

College logo

Department logo

AI-Driven Personalized Learning and Remedial Recommendation Through Knowledge Concept-Centric Evaluation

A Project Report submitted in partial fulfilment of the requirements for the award of the degree
of

Bachelor of Technology in [Computer Science and Engineering]

Submitted by

[Student Name 1] ([Roll Number])

[Student Name 2] ([Roll Number])

[Student Name 3] ([Roll Number])

Under the guidance of [Guide Name], [Designation], [Department]

[Department of Computer Science and Engineering]

[College Name]

[Affiliated University], [City]

Academic Year [2025 - 2026]

Certificate

This is to certify that the project report entitled "**AI-Driven Personalized Learning and Remedial Recommendation Through Knowledge Concept-Centric Evaluation**" is a bonafide record of the work carried out by [Student Name 1] ([Roll Number]), [Student Name 2] ([Roll Number]) and [Student Name 3] ([Roll Number]) in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology in [Computer Science and Engineering]** of [Affiliated University] during the academic year [2025 - 2026], under my supervision and guidance.

The results embodied in this report have not been submitted to any other university or institute for the award of any degree or diploma.

Project Guide

[Guide Name]

[Designation]

Head of the Department

[HOD Name]

[Department]

Principal / Director

[Principal Name]

[College Name]

Internal Examiner

External Examiner

Declaration

We hereby declare that the project work entitled "**AI-Driven Personalized Learning and Remedial Recommendation Through Knowledge Concept-Centric Evaluation**" submitted to **[College Name]**, affiliated to **[Affiliated University]**, is a record of original work done by us under the guidance of **[Guide Name]**, [Designation], [Department]. The project implements and evaluates the framework proposed in the research paper by N. Pradeesh et al. (IEEE Access, 2025) [1]; all ideas taken from that paper and from other sources are acknowledged and cited in the references. This work has not been submitted to any other university or institute for the award of any degree or diploma.

Place: [City]
Date: [DD/MM/YYYY]

Signature of the students

[Student Name 1]
[Student Name 2]
[Student Name 3]

Acknowledgement

We express our sincere gratitude to our project guide, **[Guide Name]**, [Designation], [Department], for the continuous guidance, encouragement and valuable suggestions given throughout this project.

We are thankful to **[HOD Name]**, Head of the Department of [Computer Science and Engineering], for providing the facilities and support needed to carry out this work, and to **[Principal Name]**, Principal of [College Name], for providing an excellent academic environment.

We thank all the faculty members and the technical staff of the department for their help during the project, and the project review committee for their feedback in each review.

We acknowledge the authors of the research paper "AI-Driven Personalized Learning and Remedial Recommendation Through Knowledge Concept-Centric Evaluation" (N. Pradeesh, M. G. Thushara, K. Arun Krishna, V. Pranav and S. Krishnamoorthy, IEEE Access, 2025), on which this project is based, and the developers of the open-source libraries used in this work: PyTorch, sentence-transformers, FAISS, FastAPI and pypdf.

Finally, we thank our team members [Team member names, if applicable], our families and our friends for their constant support and motivation.

[Student Name 1]
[Student Name 2]
[Student Name 3]

Abstract

Traditional learning and assessment systems give the same study material and the same test to every student and report a single total mark. A total mark does not show *which* concepts a student has not understood, and teachers of large classes cannot prepare individual remedial material for every student. This project designs and implements an AI-driven personalized learning system that evaluates learners at the level of individual knowledge concepts and closes their knowledge gaps through targeted remediation, following the framework proposed by Pradeesh et al. (IEEE Access, 2025).

The system extracts the knowledge concepts of a course from its curriculum document and stores the curriculum in a vector database for Retrieval-Augmented Generation (RAG). Learners' Outcome-Based Education (OBE) exam results are passed to an Exercise-aware Knowledge Tracing (EKT) model, a Long Short-Term Memory (LSTM) network that reads both the learner's responses and the text of the questions, and estimates the mastery of every concept. Concepts below the OBE target of 70% are flagged as knowledge gaps. For every gap, a generative AI model writes a five-part remedial recommendation, which is cross-verified by two other models, and new curriculum-based multiple-choice questions are generated, de-duplicated and verified for Bloom's Taxonomy level. An adaptive test (Easy, Medium, Hard) re-assesses the learner and its answers are fed back into knowledge tracing, forming a continuous learning loop. The system is implemented in Python with PyTorch, sentence-transformers, FAISS, FastAPI and a web frontend, and works with OpenAI, Gemini and DeepSeek models or with an offline fallback.

On a synthetic dataset for the course "Procedural Programming using C" (262 learners, 16 questions, 5 concepts), EKT reached a test AUC of 0.827 against 0.665 for the DKT baseline. In a demonstration, a learner with five knowledge gaps reached the target on every concept after one round of remediation, and in a simulated study with 30 learners the mean scores rose on all five concepts. The results show that concept-level evaluation combined with AI-generated remediation can deliver scalable, personalized support in line with OBE and Sustainable Development Goal 4.

Keywords: personalized learning, knowledge tracing, EKT, retrieval-augmented generation, remedial recommendation, adaptive assessment, outcome-based education, generative AI.

Table of Contents

Certificate	i
Declaration	ii
Acknowledgement	iii
Abstract	iv
Table of Contents	v
List of Figures	ix
List of Tables	x
List of Abbreviations	xii
1. Introduction	1
1.1 Background	1
1.1.1 Online learning	1
1.1.2 Personalized learning	1
1.1.3 Artificial intelligence in education	1
1.1.4 Adaptive assessment	1
1.1.5 Knowledge tracing	2
1.1.6 Remedial learning	2
1.2 Problem Statement	2
1.3 Motivation	2
1.4 Objectives	3
1.5 Scope of the Project	3
1.6 Relationship to the Base Research Paper	4
1.7 Organisation of the Report	4
2. Literature Review	6
2.1 Personalized Learning and AI in Education	6
2.2 Knowledge Tracing	6
2.3 Automatic MCQ Generation and RAG	6
2.4 Recommender Systems for Educational Interventions	7
2.5 Outcome-Based Education	7

2.6 Comparison of Existing Work	7
2.7 Research Gap	8
3. Existing and Proposed System	9
3.1 Existing System	9
3.2 Proposed System	10
3.3 Advantages of the Proposed System over the Existing System	12
3.4 Feasibility Study	12
4. Requirements and System Analysis	13
4.1 Functional Requirements	13
4.2 Non-Functional Requirements	13
4.3 Hardware Requirements	14
4.4 Software Requirements	14
4.5 Users and Use Cases	15
4.6 Technologies Used	15
5. System Architecture and Design	16
5.1 Overall Architecture	16
5.2 Module Decomposition	17
5.3 Data Design	17
5.4 Data Flow of One Learning Round	18
5.5 API Design	19
5.6 Configuration	19
6. Methodology	20
6.1 Overall Algorithm	20
6.2 Module 1: Course Material Processing	20
6.3 Module 2: Retrieval-Augmented Generation	21
6.4 Concept Extraction	22
6.5 MCQ Generation	23
6.6 Student Assessment Data	24
6.7 Knowledge Tracing with EKT	24
6.8 Knowledge Gap Identification	26
6.9 Personalized Remedial Recommendation	27
6.10 Adaptive Evaluation and Re-testing	27

6.11 Continuous Learning Loop	28
6.12 Evaluation Metrics	29
7. Implementation	30
7.1 Development Environment	30
7.2 Project Structure	30
7.3 Implementation of the Modules	31
7.3.1 LLM providers and offline fallback	31
7.3.2 Knowledge tracing	31
7.3.3 Gap analysis, recommendations, questions and evaluation	31
7.4 Command-Line Interface	32
7.5 User Interface	32
7.6 Testing	39
7.7 Problems Encountered and Solutions	40
8. Results and Evaluation	42
8.1 Experimental Setup	42
8.2 Knowledge Tracing Results	43
8.3 Question Generation Results	44
8.4 Knowledge Gap Analysis of the Class	45
8.5 Case Study: Learning Loop of One Learner	46
8.6 Before vs After Analysis	47
8.7 User Evaluation	48
8.8 Comparison with the Results Reported in the Paper	48
8.9 Discussion	49
9. Advantages, Limitations and Future Scope	50
9.1 Advantages	50
9.2 Limitations	50
9.3 Future Scope	50
10. Conclusion	52
References	53
Appendices	55
Appendix A: Sample Prompts	55
Appendix B: Sample Generated MCQs	56

Appendix C: Sample Remedial Recommendation	57
Appendix D: Data Formats	58
Appendix E: API Request and Response Example	59
Appendix F: Configuration File (config.yaml)	60
Appendix G: Test Execution Output	60
Appendix H: Learner Questionnaire	61
Appendix I: Installation and User Manual	61
Appendix J: Sample Command-Line Output	62

List of Figures

Figure 3.1: Existing system	9
Figure 3.2: Proposed system: the continuous personalized learning loop	11
Figure 5.1: Layered system architecture	16
Figure 5.2: Data entities and relationships	17
Figure 6.1: Retrieval-Augmented Generation pipeline	22
Figure 6.2: Concept hierarchy of the sample course	22
Figure 6.3: MCQ generation, de-duplication and verification pipeline	23
Figure 6.4: EKT model architecture as implemented	25
Figure 6.5: Remedial recommendation with cross-verification	27
Figure 6.6: Adaptive evaluation and resulting categories	28
Figure 7.1: Home page: loading a sample learner, uploading a CSV or entering results manually	33
Figure 7.2: Exam results of learner CSE230001 loaded and editable	33
Figure 7.3: Uploading an OBE results CSV	34
Figure 7.4: Knowledge state: EKT mastery per concept with threshold and target	34
Figure 7.5: Personalized remedial recommendation with its five sections	35
Figure 7.6: Adaptive self-assessment: generated questions at the Easy level	36
Figure 7.7: Graded level with correct and wrong answers and explanations	37
Figure 7.8: Self-assessment completed for all weak concepts	38
Figure 7.9: Round 2: updated knowledge state and progress across rounds	39
Figure 7.10: Interactive REST API documentation (/docs)	39
Figure 8.1: Test-set metrics of EKT and DKT	43
Figure 8.2: AUC over training epochs	44
Figure 8.3: Loss over training epochs	44
Figure 8.4: Generated, duplicate and accepted questions	45
Figure 8.5: Share of learners attaining each concept	46
Figure 8.6: Mastery of learner CSE230001 in rounds 1 and 2	46
Figure 8.7: Mean score per concept before and after remediation	47
Figure 8.8: Concept-level comparison before and after remediation	47
Figure 8.9: Distribution of competency levels before and after remediation	48

List of Tables

Table 1.1: Scope of the project	3
Table 1.2: What the paper proposes and what this project implements	4
Table 2.1: Comparison of existing systems and the proposed approach	7
Table 3.1: Existing system compared with the proposed system	12
Table 4.1: Functional requirements	13
Table 4.2: Hardware requirements	14
Table 4.3: Software requirements (versions used)	14
Table 4.4: Users and their use cases	15
Table 4.5: Technologies used	15
Table 5.1: Modules with their inputs and outputs	17
Table 5.2: Logical data design and where each entity is stored	18
Table 5.3: Sequence of one learning round	18
Table 5.4: REST API endpoints	19
Table 5.5: Main configuration parameters (config.yaml)	19
Table 6.1: Extracted concepts of the sample course	23
Table 6.2: Difficulty levels and Bloom's Taxonomy levels	24
Table 6.3: Example question records from the generated question bank	24
Table 6.4: OBE records of learner CSE230001 (sample data)	24
Table 6.5: Classification rules	26
Table 6.6: Gap analysis of learner CSE230001 before remediation (round 1)	26
Table 6.7: Evaluation metrics	29
Table 7.1: Commands of the command-line interface (python -m plrs)	32
Table 7.2: Test cases	40
Table 7.3: Problems found during development and how they were solved	41
Table 8.1: Datasets used in this project	42
Table 8.2: Experimental environment	43
Table 8.3: Knowledge tracing performance on the test set (after 12 epochs)	43
Table 8.4: Knowledge tracing performance on the training set	43
Table 8.5: Question generation pipeline statistics and quality metrics	45
Table 8.6: Knowledge gap matrix of the current batch (target 70%)	45
Table 8.7: EKT mastery of learner CSE230001 before and after one round of remediation	46
Table 8.8: Paired t-test of scores before and after remediation (simulated learners)	47

Table 8.9: Our results compared with the paper's reported results	48
Table A.1: Learner questionnaire	61
Table A.2: Knowledge-gap analysis output for learner CSE230001	62

List of Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
AUC	Area Under the ROC Curve
BCE	Binary Cross-Entropy
BKT	Bayesian Knowledge Tracing
BTL	Bloom's Taxonomy Level
CLI	Command-Line Interface
CO	Course Outcome
CSV	Comma-Separated Values
DKT	Deep Knowledge Tracing
DKVMN	Dynamic Key-Value Memory Network
EKT	Exercise-aware Knowledge Tracing
FAISS	Facebook AI Similarity Search
IRT	Item Response Theory
JSON	JavaScript Object Notation
KT	Knowledge Tracing
LLM	Large Language Model
LMS	Learning Management System
LSTM	Long Short-Term Memory
MAE	Mean Absolute Error
MCQ	Multiple-Choice Question
NLP	Natural Language Processing
OBE	Outcome-Based Education
PDF	Portable Document Format
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
ROC	Receiver Operating Characteristic
S-BERT	Sentence-BERT
SDG	Sustainable Development Goal
UI	User Interface

CHAPTER 1

Introduction

1.1 Background

Education has moved rapidly towards digital and online learning. Learning Management Systems (LMS) now hold course material, conduct examinations and record every answer a student gives. This creates an opportunity: the same data that is used to calculate marks can also be used to understand *how well each student knows each topic*, and to help each student individually.

1.1.1 Online learning

Online learning platforms deliver lectures, notes and tests through the web. They make learning available at any time and place, and they record detailed interaction data such as which questions a learner attempted and whether each answer was correct. In most platforms, however, every learner still receives the same content in the same order.

1.1.2 Personalized learning

Personalized learning adapts content, pace and assessment to the needs of an individual learner. A learner who already understands a topic can move ahead, while a learner who struggles receives more explanation and practice. Personalization is known to improve engagement and learning outcomes, but doing it manually for every student in a large class is not practical for a teacher.

1.1.3 Artificial intelligence in education

Artificial intelligence makes personalization scalable. Machine-learning models can predict a learner's performance from their past answers, and generative AI models such as large language models (LLMs) can write explanations, examples and questions in natural language. Combined, they can act as an assistant that diagnoses weaknesses and prepares individual learning material automatically [13], [14].

1.1.4 Adaptive assessment

In an adaptive assessment the next question depends on the learner's previous answers. A common pattern is to start with easy questions and move to harder ones only when the learner succeeds. This gives a more precise picture of the learner's level than a fixed test, with fewer questions.

1.1.5 Knowledge tracing

Knowledge tracing (KT) is the task of estimating, after every interaction, how well a learner has mastered each knowledge concept. Early work used Bayesian Knowledge Tracing [25]; deep-learning models such as Deep Knowledge Tracing (DKT) [3] use recurrent neural networks, and Exercise-aware Knowledge Tracing (EKT) [2] also reads the text of each exercise. The output of a KT model is a knowledge state: one mastery probability per concept.

1.1.6 Remedial learning

Remedial learning is additional, targeted instruction for a learner who has not reached the expected level. Effective remediation focuses on the specific concepts the learner is missing, explains them again in a different way, and provides practice, instead of repeating the whole course.

Different students have different levels of understanding of individual concepts. A conventional assessment system generally evaluates overall marks but does not clearly identify which specific concepts a learner has not mastered. This project addresses that gap.

1.2 Problem Statement

Existing learning systems often provide the same learning materials and assessments to all learners without considering their individual concept-level knowledge. Overall marks do not reveal which concepts a learner is weak in, and preparing individual remedial material manually does not scale. Therefore, there is a need for an intelligent system that identifies knowledge gaps at the concept level, provides personalized remedial learning recommendations, re-assesses the learner with new questions, and repeats this process until the learner reaches the required level of competency.

1.3 Motivation

- **Students learn differently.** Learners in the same class understand concepts at different speeds and have different backgrounds.
- **Marks alone do not reveal gaps.** A score of 60% can hide a complete lack of understanding of one concept.
- **Teachers cannot personalize manually.** Preparing individual material and tests for every student in a large class is not feasible.
- **AI can generate personalized content.** Retrieval-Augmented Generation can produce explanations and questions that stay within the syllabus.
- **Continuous evaluation shows improvement.** Knowledge tracing can track how each learner improves after each intervention.

- **Outcome-Based Education requires measurable attainment.** OBE defines a threshold and a target for every course outcome, which gives a clear, measurable definition of a knowledge gap.
- **Social relevance.** Personalized, equitable support for every learner contributes to Sustainable Development Goal 4, quality education for all [31].

1.4 Objectives

1. Extract the knowledge concepts of a course from its curriculum document.
2. Build a searchable knowledge base of the curriculum using Retrieval-Augmented Generation (RAG) and a vector database.
3. Generate concept-aligned multiple-choice questions at the required difficulty and Bloom's Taxonomy level.
4. Evaluate learner responses and store them as OBE exam records.
5. Estimate the concept-level mastery of each learner with an Exercise-aware Knowledge Tracing (EKT) model and compare it with a Deep Knowledge Tracing (DKT) baseline.
6. Identify weak concepts using the OBE threshold and target.
7. Generate personalized, cross-verified remedial recommendations for every weak concept.
8. Generate new, non-duplicate questions for re-assessment through an adaptive (Easy, Medium, Hard) test.
9. Track improvement before and after remediation, and evaluate it statistically.
10. Provide a web interface for learners and teachers, a REST API, and a command-line interface.

1.5 Scope of the Project

Table 1.1: Scope of the project

Included	Not included
Processing of curriculum PDF / text documents	Full replacement of the teacher
Extraction of knowledge concepts (reviewed by the teacher)	Automatic grading of subjective (descriptive) answers; the system uses MCQs and marks
Generation of MCQs with difficulty and Bloom's level	Complete LMS deployment with login, courses and enrolment; the project is a working prototype
Concept-level knowledge tracing (EKT, DKT)	Video or multimedia content generation
Knowledge-gap detection with OBE threshold and target	Evaluation on real institutional student data (synthetic data is used)
Personalized remedial recommendations with cross-verification	A controlled classroom study with real learners
Adaptive re-testing and the continuous learning loop	
Result visualisation, experiments and statistical tests	
Web frontend, REST API and command-line interface	

1.6 Relationship to the Base Research Paper

This project is based on the research paper "AI-Driven Personalized Learning and Remedial Recommendation Through Knowledge Concept-Centric Evaluation" by N. Pradeesh et al., published in IEEE Access in 2025 [1]. The paper proposes the framework; this project is our own implementation of it. To keep the report honest, the two are clearly separated below.

Table 1.2: What the paper proposes and what this project implements

Component	Research paper [1]	This project
Curriculum processing and RAG	PDF to text, embeddings, vector database	Implemented (pypdf, all-MiniLM-L6-v2, FAISS), with unit-aware chunking added
Concept extraction	Generative AI, reviewed by instructors	Implemented; concepts saved to concepts.json for teacher review
Knowledge tracing	EKT; compared with DKT, DKVMN, AKT, SimpleKT	EKT and DKT implemented and compared; DKVMN, AKT, SimpleKT not implemented
Gap analysis	OBE threshold and target	Implemented
Remedial recommendation	OpenAI, five sections, cross-verified by Gemini and DeepSeek	Implemented with the same design
MCQ generation	RAG, 3x generation, de-duplication (0.85), cross-verification	Implemented with the same design
Adaptive evaluation	Easy, Medium, Hard with instructor rules	Implemented
Data	Real OBE data from the AMPLE LMS (1,276 learners)	Synthetic data from an IRT-based learner simulator
Learner survey	Conducted (Table 7 of the paper)	Not conducted; questionnaire provided in Appendix H
Interfaces	In-house LMS (AMPLE)	Web frontend, REST API and CLI built for this project

1.7 Organisation of the Report

- **Chapter 2** reviews the literature on personalized learning, knowledge tracing, question generation and educational recommender systems, and identifies the research gap.
- **Chapter 3** describes the existing system and the proposed system.
- **Chapter 4** lists the functional and non-functional requirements, hardware and software, and technologies.
- **Chapter 5** presents the system architecture, module decomposition, data design and API design.
- **Chapter 6** explains the methodology of every module in detail.
- **Chapter 7** describes the implementation, the user interface, testing and the problems solved during development.
- **Chapter 8** presents the experimental setup, results and evaluation.

- **Chapter 9** discusses advantages, limitations and future scope, and **Chapter 10** concludes the report.
- The **Appendices** contain sample prompts, generated questions, a recommendation, data formats, API examples, the configuration, test cases, the questionnaire and a user manual.

CHAPTER 2

Literature Review

2.1 Personalized Learning and AI in Education

AI-based personalized e-learning systems adapt content and assessment to learner profiles using intelligent tutoring, adaptive platforms and predictive analytics [13]. Peng et al. [14] describe personalized adaptive learning as an approach in which a smart learning environment adjusts to individual characteristics and performance. Large reviews of AI in higher education [28], [29] report strong potential but also call for more transparency, explainability and ethical use. Most systems, however, adapt to learning style or overall performance, and rarely perform fine-grained concept-level gap analysis aligned with course outcomes [1].

2.2 Knowledge Tracing

Knowledge tracing estimates a learner's mastery of concepts over time. Bayesian Knowledge Tracing [25] models each concept as a hidden binary state. Deep Knowledge Tracing (DKT) [3] replaced this with a recurrent neural network over the sequence of (concept, correctness) pairs and improved prediction accuracy. DKVMN [4] uses a key-value memory to model each concept separately, AKT [5] adds context-aware attention, and SimpleKT [6] shows that a simple model can be a strong baseline. EKT [2] integrates the text of each exercise with the response sequence, so the model knows *what* was asked, not only which concept it belongs to. The base paper selected EKT after it achieved the highest AUC among these models on its data [1].

2.3 Automatic MCQ Generation and RAG

Retrieval-Augmented Generation [7], [8] combines a retriever, which finds relevant passages in a document collection, with a generative model that writes the answer using those passages. For education this keeps generated content inside the syllabus. MCQGen [9] uses LLMs with RAG and prompting strategies such as chain-of-thought and self-refinement to generate MCQs for personalized learning. Generating valid questions with believable distractors and ensuring pedagogical suitability and Bloom's-level alignment [11] remain open problems, and few systems verify generated questions with independent models [1], [27].

2.4 Recommender Systems for Educational Interventions

Educational recommender systems use collaborative filtering, content-based filtering, knowledge-based methods and hybrids to suggest learning resources [15]. Deep-learning and reinforcement-learning recommenders adapt to evolving learner states. These systems mostly match learners with items and seldom track how the knowledge state of each concept changes over time or close the loop with re-assessment [1].

2.5 Outcome-Based Education

Outcome-Based Education [12] designs courses around measurable course outcomes (COs), each linked to knowledge concepts and mapped to program outcomes. Attainment is measured against a threshold (minimum acceptable level) and a target (desired level). This gives a precise, institution-approved definition of when a learner has or has not achieved a concept, which this project uses to define knowledge gaps.

2.6 Comparison of Existing Work

Table 2.1: Comparison of existing systems and the proposed approach

Paper / system	Method	Advantages	Limitations
BKT, Corbett & Anderson [25]	Hidden Markov model per concept	Simple, interpretable	One concept at a time; no exercise content
DKT, Piech et al. [3]	RNN/LSTM over (concept, correct) sequence	Higher accuracy; learns concept relations	Ignores exercise text; mastery hard to interpret
DKVMN, Zhang et al. [4]	Key-value memory network	Separate state per concept	No exercise text
AKT, Ghosh et al. [5]	Context-aware attention	Models forgetting and context	More complex, needs more data
EKT, Liu et al. [2]	Exercise text embedding + LSTM + attention	Uses what was asked; strong accuracy	Needs exercise text
MCQGen, Hang et al. [9]	LLM + RAG MCQ generation	Context-relevant questions	No knowledge tracing or remediation loop
Educational recommenders [15]	Collaborative / content-based / hybrid	Personalized resources	Limited concept-level tracking over time
Pradeesh et al. [1]	RAG + EKT + generative AI + cross-verification in an OBE loop	Integrated, concept-level, closed loop	Many components; depends on LLM quality and data
This project	Implementation of [1] with web UI, offline mode, tests	Runs end to end; reproducible	Synthetic data; two KT models only

2.7 Research Gap

Previous work addresses adaptive assessment, question generation and recommendation separately, and rarely integrates them with Outcome-Based Education. Existing validation methods seldom cross-verify generated remedial content with different AI models. The base paper [1] addresses this by combining EKT-based continuous knowledge tracing, RAG-based assessment and cross-verified remedial recommendation in one loop. The gap this project fills is a complete, working and tested implementation of that integrated loop, usable through a web interface, which can run with real AI models or fully offline.

CHAPTER 3

Existing and Proposed System

3.1 Existing System

In the existing approach, every student studies the same material and takes the same test. The result is a total mark and, at best, generic feedback such as "revise chapter 3".

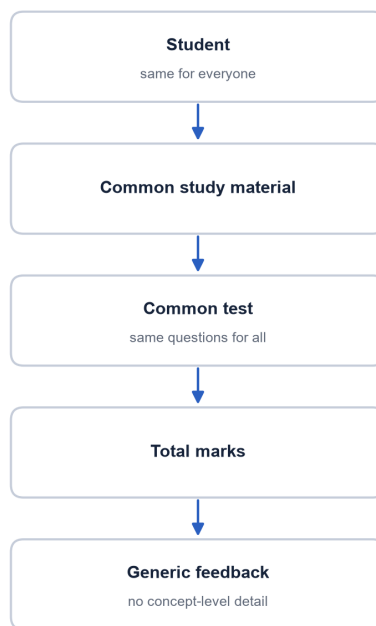


Figure 3.1: Existing system

Problems with the existing system:

- The same content is given to everyone, regardless of their individual understanding.
- Overall marks do not show which individual concepts are weak.
- Personalization is limited, and preparing individual material manually is time-consuming.
- There is no continuous tracking of knowledge over time.
- No targeted remedial material is produced, and re-tests usually repeat the same questions.

3.2 Proposed System

The proposed system replaces the single mark with a concept-level knowledge state and closes the loop between assessment, remediation and re-assessment. Figure 3.2 shows the complete flow. The teacher uploads the curriculum once; the system extracts concepts and builds the RAG vector database. For each learner, exam results are traced by EKT, weak concepts are identified, a personalized study plan and new questions are generated, and an adaptive re-test measures progress. The re-test answers are fed back into knowledge tracing, and the loop repeats until every concept reaches the OBE target.

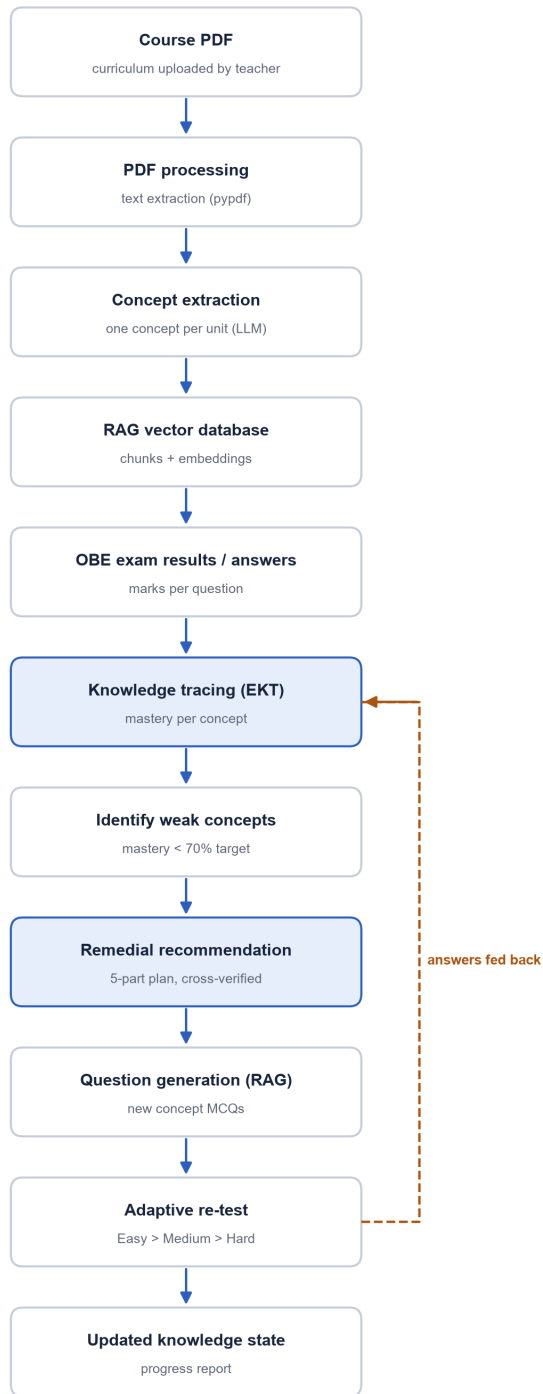


Figure 3.2: Proposed system: the continuous personalized learning loop

3.3 Advantages of the Proposed System over the Existing System

Table 3.1: Existing system compared with the proposed system

Aspect	Existing system	Proposed system
Unit of evaluation	Total marks	Mastery of each concept
Content	Same for everyone	Personalized study plan per weak concept
Questions	Fixed, written manually	Generated from the syllabus, de-duplicated, verified
Test	Same test for all	Adaptive: Easy, Medium, Hard, only on weak concepts
Tracking	One snapshot	Continuous knowledge tracing after every answer
Feedback	Generic	Explains which gaps were found and why the plan was given
Teacher effort	High	Teacher reviews concepts and flagged items only

3.4 Feasibility Study

- **Technical feasibility:** all components use mature open-source libraries (PyTorch, sentence-transformers, FAISS, FastAPI) and run on an ordinary laptop CPU; no GPU is required.
- **Economic feasibility:** the software is free and open source. Commercial LLM APIs are optional and billed per use; the offline mode costs nothing.
- **Operational feasibility:** teachers upload a PDF and a results CSV they already have; learners use a web page that needs no installation.

CHAPTER 4

Requirements and System Analysis

4.1 Functional Requirements

Table 4.1: Functional requirements

ID	Requirement	Module
FR1	Upload a curriculum PDF or text file and index it in a vector database	rag.py, api.py
FR2	Extract knowledge concepts with course outcomes and learning objectives	rag.py
FR3	Allow the teacher to review and edit the concept list	concepts.json
FR4	Accept OBE exam results as CSV, from sample data or typed in	data.py, api.py, frontend
FR5	Train EKT and DKT knowledge-tracing models on historical data	kt.py
FR6	Estimate mastery of every concept for a learner	kt.py
FR7	Classify each concept as Beginner, Intermediate or Expert and flag gaps	gap.py
FR8	Generate a five-part remedial recommendation for every gap	remedial.py
FR9	Cross-verify recommendations and questions with secondary models	remedial.py, qgen.py
FR10	Generate MCQs per concept, difficulty and Bloom's level; remove duplicates	qgen.py
FR11	Run an adaptive Easy/Medium/Hard test with an instructor-defined rule	evaluation.py
FR12	Feed test answers back into knowledge tracing and repeat the loop	pipeline.py
FR13	Report results: gap matrix, KT metrics, question metrics, pre/post t-test	experiments.py
FR14	Provide a web frontend, a REST API and a command-line interface	static, api.py, cli.py

4.2 Non-Functional Requirements

- **Reproducibility:** fixed random seeds; every setting is in one file, config.yaml.
- **Robustness:** if an AI provider has no key, no credit or is unreachable, the system falls back to an offline mock that returns the same JSON format, so it never stops working.
- **Performance:** one EKT training epoch on 24,000 answers takes about 2-3 seconds on a laptop CPU; analysis of a learner takes milliseconds after the models are loaded.
- **Usability:** a single-page web interface with no installation for learners.
- **Security and privacy:** API keys are kept in a .env file excluded from version control; answers are not shown to the learner before submission; learner records can be anonymised.
- **Maintainability:** modular Python package with 30 automated tests.
- **Portability:** runs on Windows, Linux and macOS with Python 3.10 or newer.

4.3 Hardware Requirements

Table 4.2: Hardware requirements

Component	Minimum	Used in this project
Processor	Dual-core 64-bit CPU	13th Gen Intel Core i5-1335U (10 cores, 12 threads)
Memory (RAM)	8 GB	15.7 GB
Storage	3 GB free (Python packages and embedding model)	SSD
GPU	Not required	Not used (CPU-only PyTorch)
Network	Only for first model download and optional AI APIs	Broadband

4.4 Software Requirements

Table 4.3: Software requirements (versions used)

Software	Version	Purpose
Operating system	Windows 11 Pro (also runs on Linux/macOS)	Development and testing
Python	3.13.2 (3.10+ supported)	Implementation language
PyTorch	2.14 (CPU)	EKT and DKT neural networks
NumPy	2.5	Numerical computing
sentence-transformers	6.1 (model all-MiniLM-L6-v2)	Text embeddings
FAISS (faiss-cpu)	1.15	Vector similarity search
pypdf	6.19	Reading curriculum PDFs
FastAPI / Uvicorn	0.141 / 0.54	REST API and web server
openai, google-generativeai	3.20 / 0.8	OpenAI, DeepSeek and Gemini clients
SciPy, sacrebleu	1.18 / 2.6	t-test, Self-BLEU (optional; built-in fallbacks exist)
matplotlib	3.11	Charts
pytest	9.1	Automated tests
Web browser	Microsoft Edge / Chrome / Firefox	Web frontend

4.5 Users and Use Cases

Table 4.4: Users and their use cases

User	Use cases
Teacher / Admin	Upload curriculum; review concepts; upload OBE results; set the evaluation rule; view gap matrix and experiment results
Learner	View knowledge state and gaps; read personalized recommendations; take the adaptive self-assessment; see progress across rounds
Developer / Researcher	Train and compare KT models; run experiments; use the REST API; run tests

4.6 Technologies Used

Only technologies that are actually used in the implementation are listed.

Table 4.5: Technologies used

Technology	Purpose in this project
Python	Backend, AI pipeline and experiments
PyTorch	EKT and DKT models (LSTM, attention, training)
sentence-transformers (all-MiniLM-L6-v2)	Embeddings of curriculum chunks, questions and exercise texts
FAISS	Vector database for the curriculum (numpy fallback when not installed)
pypdf	Extracting text from curriculum PDFs
OpenAI GPT-4o-mini	Primary generator: concepts, recommendations, MCQs
Google Gemini, DeepSeek	Cross-verification of recommendations and MCQs
FastAPI + Uvicorn	REST API and hosting of the web frontend
HTML, CSS, JavaScript	Single-page web frontend
JSON, CSV and NumPy files	Data storage (concepts, question bank, records, vectors); no SQL database is used
PyYAML, python-dotenv	Configuration and API key loading
matplotlib	Result charts
pytest	Automated testing

CHAPTER 5

System Architecture and Design

5.1 Overall Architecture

The system is organised in five layers (Figure 5.1). Users interact through the web frontend, the REST API or the command line. All three call the pipeline, which runs the six functional modules in the order of the paper's Algorithm 1. The modules share a foundation layer: LLM providers, embeddings, the OBE data layer, metrics and configuration.



Figure 5.1: Layered system architecture

5.2 Module Decomposition

Table 5.1: Modules with their inputs and outputs

Module (file)	Input	Output
Curriculum RAG (rag.py)	Curriculum PDF/TXT	Vector DB of chunks; concepts.json
Knowledge tracing (kt.py)	Sequence of OBE records	Mastery per concept (knowledge state)
Gap analysis (gap.py)	Knowledge state, OBE scores	Level and gap flag per concept
Remedial recommendation (remedial.py)	Weak concept, attempts, curriculum context	Five-part, cross-verified plan
MCQ generation (qgen.py)	Concept, level, curriculum context	Verified, unique MCQs in the question bank
Adaptive evaluation (evaluation.py)	Weak concepts, rule, question bank	Category per concept; new OBE records
Pipeline (pipeline.py)	Learner records	Rounds of state, gaps, plans and results
Interfaces (api.py, cli.py, static/)	User actions	Pages, JSON responses, console output

5.3 Data Design

The prototype stores its data in files rather than in a relational database, which keeps it portable and easy to inspect. Figure 5.2 shows the data entities and their relationships; Table 5.2 maps them to the logical tables a database version would use.

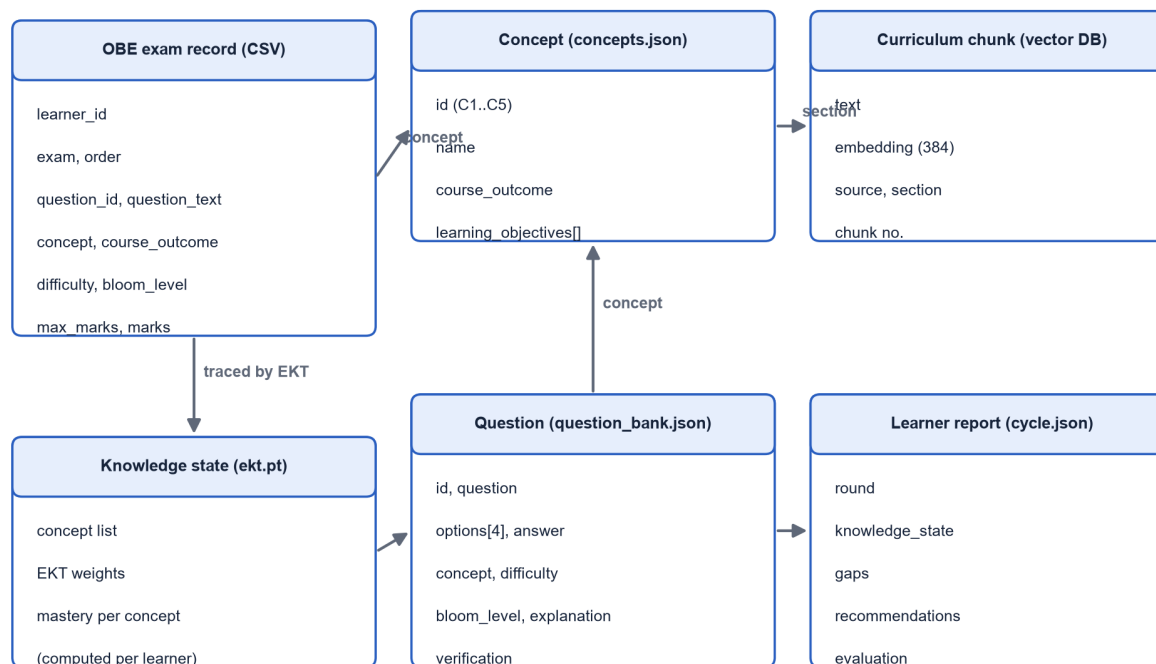


Figure 5.2: Data entities and relationships

Table 5.2: Logical data design and where each entity is stored

Entity (logical table)	Fields	Storage in this project
Student / Learner	learner_id	learner_id column of the OBE CSV
Concept	id, name, course_outcome, learning_objectives	data/store/concepts.json
Curriculum chunk	text, embedding, source, section, chunk	data/store/curriculum.npy + curriculum.json
Question	id, question, options, answer, concept, difficulty, bloom_level, explanation, verification	data/store/question_bank.json
Student response (OBE record)	learner_id, exam, order, question_id, question_text, concept, course_outcome, difficulty, bloom_level, max_marks, marks	data/sample/*.csv or uploaded CSV
Knowledge state	learner, concept, mastery	Computed by data/models/ekt.pt; saved per round in cycle.json
Recommendation	learner, concept, 5 sections, verification, round	data/store/learners/<id>/cycle.json and cycle.md

5.4 Data Flow of One Learning Round

Table 5.3: Sequence of one learning round

Step	Component	Action
1	Frontend / CLI	Sends the learner's OBE records to /analyze
2	kt.py	EKT reads the sequence and returns mastery per concept
3	gap.py	Compares mastery with the 70% target; returns levels and gaps
4	remedial.py	For each gap: retrieves context, generates the plan, cross-verifies it
5	qgen.py	Ensures enough unused questions exist for each level; generates more if needed
6	evaluation.py	Serves Easy questions, grades them, unlocks Medium and Hard
7	pipeline.py	Appends the answers to the history and repeats from step 2

5.5 API Design

Table 5.4: REST API endpoints

Method	Path	Purpose
GET	/	Web frontend
GET	/health	Active models, KT model, OBE threshold and target
POST	/curriculum	Upload a curriculum file; build the vector DB; extract concepts
GET	/concepts	List concepts
GET	/sample-learners, /sample-learners/{id}	Sample learners and their records
POST	/results/upload	Upload an OBE results CSV
POST	/analyze	Knowledge state, levels and gaps for a learner
POST	/recommend	Remedial recommendations for the learner's gaps
POST	/mcq	Generate questions for a concept and level
POST	/evaluation	Start an adaptive evaluation session
POST	/evaluation/{id}/answer	Submit answers for one level; get the next level
GET	/evaluation/{id}	Session report including answers as OBE records

5.6 Configuration

Table 5.5: Main configuration parameters (config.yaml)

Parameter	Value	Source
OBE threshold / target	50% / 70%	Paper Sec. III-A.1
Chunk size / overlap / top-k	1000 / 100 characters / 4	Paper Appendix A
Embedding model	sentence-transformers/all-MiniLM-L6-v2	Paper Sec. V-A
MCQ generation multiplier	3	Paper Appendix A
De-duplication threshold	0.85 cosine similarity	Paper Sec. III-D.1
Verification minimum score	3 out of 5	This project
Default evaluation rule	5 questions per level, 80% to pass	Paper Sec. IV-A.4
EKT embedding / LSTM / dropout	100 / 100 / 0.3	Paper Sec. V-B
Optimizer / learning rate / batch / epochs	Adam / 0.001 / 32 / 12	Paper Sec. V-B
Train/test split	75% / 25% of learners	Paper Sec. V-B
Primary LLM / verifiers	OpenAI / Gemini, DeepSeek	Paper Sec. V-A.1

CHAPTER 6

Methodology

6.1 Overall Algorithm

The complete method follows Algorithm 1 of the base paper [1]. It is implemented in pipeline.py.

Algorithm 1 (from [1], as implemented)

```

Algorithm 1: Personalized Learning and Recommendation
Input : course curriculum (PDF), OBE exam results
Output: remedial recommendations and personalized evaluation

Step 1 Data preparation
    1.1 Convert curriculum PDF to text
    1.2 Split into chunks, embed, store in the vector database; extract concepts
Step 2 Knowledge gap identification
    2.1 EKT knowledge tracing -> mastery per concept; mastery < target -> gap
Step 3 Remedial recommendation
    3.1 Generate a five-part plan for every gap (primary LLM)
    3.2 Cross-verify with secondary LLMs; regenerate if the score is low
Step 4 MCQ generation
    4.1 Generate concept MCQs with RAG (about 3x the number needed)
    4.2 Remove duplicates by semantic similarity (> 0.85)
    4.3 Cross-verify Bloom's level, difficulty and relevance
Step 5 Personalized evaluation
    5.1 Select questions for the weak concepts
    5.2 Build an adaptive test with the instructor-defined rule
    5.3 Deliver the test to the learner
    5.4 Pass the results back to Step 2
    5.5 Continue until every concept reaches the target

```

6.2 Module 1: Course Material Processing

Input: the curriculum document (PDF or text). **Process:** the text is extracted with pypdf, whitespace is normalised, and the document is split into sections on headings of the form "Unit N: <title>" (also Module or Chapter). Each section is split into chunks of about 1000 characters with 100 characters of overlap. Chunk boundaries are placed at sentence ends, and each new chunk starts at the beginning of a sentence. Every chunk is embedded and stored with its source file and section name. **Output:** a concept-based knowledge base.

rag.py - chunk_text()

```
def chunk_text(text: str, chunk_size: int | None = None, overlap: int | None = None) -> list[str]:
    """Character chunks (~1000) with overlap (~100), cut at sentence/line boundaries when
    possible."""
    cfg = get_config()["rag"]
    size = chunk_size or cfg["chunk_size"]
    overlap = cfg["chunk_overlap"] if overlap is None else overlap
    text = text.strip()
    boundary = re.compile(r"(?<=[.!?])\s+|\n+")
    chunks, start = [], 0
    while start < len(text):
        end = min(start + size, len(text))
        if end < len(text):
            cuts = [m.end() for m in boundary.finditer(text, start + size // 2, end)]
            if cuts:
                end = cuts[-1]
            else:
                space = text.rfind(" ", start + size // 2, end)
                end = space if space != -1 else end
            chunks.append(text[start:end].strip())
            if end >= len(text):
                break
            # next chunk starts ~overlap chars back, at the start of the sentence containing that point
            back = max(end - overlap, start + 1)
            cuts = [m.end() for m in boundary.finditer(text, start + 1, back + 1) if m.end() <= back]
            if cuts:
                start = cuts[-1]
            else:
                space = text.find(" ", back, end)
                start = space + 1 if space != -1 else end
    return [c for c in chunks if c]
```

6.3 Module 2: Retrieval-Augmented Generation

RAG ensures that everything the AI writes is based on the course material. Each chunk is converted into a 384-dimensional embedding with all-MiniLM-L6-v2 [10], [22] and stored in a FAISS inner-product index [16]. Because the vectors are normalised, the inner product equals cosine similarity:

$$\cos(a, b) = (a \cdot b) / (||a|| ||b||)$$

To build the context for a concept, the query is the concept name plus its learning objectives. Chunks from the concept's own curriculum section are ranked by similarity and used first; only if a concept has no matching section are the nearest chunks from the whole curriculum used. The retrieved text is inserted into the prompt of the generative model.



The LLM only sees retrieved syllabus text, so generated content stays inside the curriculum.

Figure 6.1: Retrieval-Augmented Generation pipeline

Why RAG is used:

- It keeps generated questions and explanations inside the actual syllabus.
- It retrieves the information specific to one concept.
- It reduces irrelevant or invented (hallucinated) content.

6.4 Concept Extraction

The primary LLM is asked to list one concept per unit, with its course outcome and learning objectives, and to return JSON. The result is saved to concepts.json so that the teacher can review and correct it, as the paper requires. For the sample course "Procedural Programming using C" the five extracted concepts are shown in Figure 6.3 and Table 6.1.

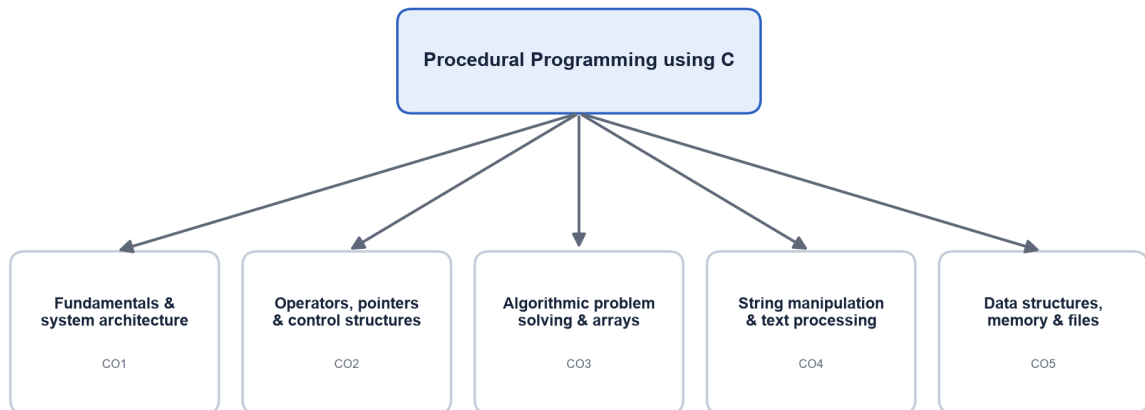


Figure 6.2: Concept hierarchy of the sample course

Table 6.1: Extracted concepts of the sample course

ID	Concept	Course outcome
C1	Fundamentals and System Architecture	CO1
C2	Operators, Pointers and Control Structures	CO2
C3	Algorithmic Problem Solving and Array Manipulation	CO3
C4	String Manipulation and Text Processing	CO4
C5	Data Structures, Memory Management, and File Handling	CO5

6.5 MCQ Generation

For a concept and a difficulty level, the generator retrieves the concept's context and asks the primary model for about three times the required number of questions. Each question has a stem, four options, the index of the correct answer, an explanation, the concept, the difficulty and the Bloom's Taxonomy level [11]. The pipeline then:

1. **Refines** each question with the same model (grammar, format, ambiguity).
2. **De-duplicates:** the question text plus its correct answer is embedded and compared with every question in the global question memory; if the maximum cosine similarity is above 0.85 the question is discarded.
3. **Cross-verifies:** Gemini and DeepSeek each score the question from 1 to 5 for concept fit, difficulty, Bloom's level and correctness of the key; questions with an average score below 3 are rejected.
4. **Stores** accepted questions in the question bank (question_bank.json), which persists across runs.

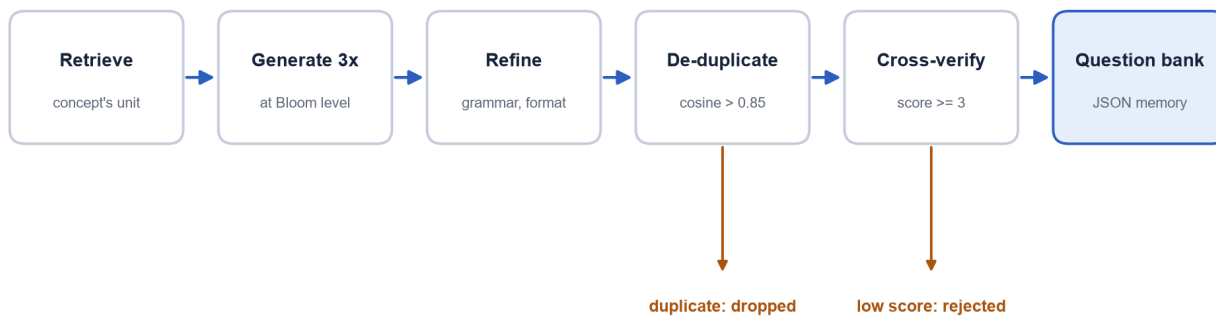
**Figure 6.3: MCQ generation, de-duplication and verification pipeline**

Table 6.2: Difficulty levels and Bloom's Taxonomy levels

Difficulty	Bloom's levels	Typical question
Easy	Remember, Understand	Definition, fill in the blank, identify the correct statement
Medium	Apply, Analyze	Apply a rule to a situation; trace code
Hard	Evaluate, Create	Find the incorrect statement; design or justify a solution

Table 6.3: Example question records from the generated question bank

Question (shortened)	Concept	Difficulty	Bloom level
Fill in the blank (Algorithmic Problem Solving and Array Manipulation): "To find the large..."	Algorithmic Problem Solving & Arrays	easy	Remember
A learner must apply ideas from Algorithmic Problem Solving and Array Manipulation to solve...	Algorithmic Problem Solving & Arrays	medium	Analyze
Which of the following statements about Algorithmic Problem Solving and Array Manipulation...	Algorithmic Problem Solving & Arrays	hard	Evaluate
Fill in the blank (Data Structures, Memory Management, and File Handling): "Dynamic memory..."	Data Structures, Memory & Files	easy	Remember

6.6 Student Assessment Data

Every answer is stored as an OBE record (one CSV row). A question counts as **attained** when the marks obtained reach the OBE threshold: $\text{marks} / \text{max_marks} \times 100 \geq 50$. The concept score is the total marks obtained divided by the total maximum marks for that concept.

Table 6.4: OBE records of learner CSE230001 (sample data)

Question	Concept	Difficulty	Max	Marks
Which function opens a file for reading in C?	Data Structures, Memory & Files	easy	2.0	0.0
Which operator is used to obtain the address of a variable?	Operators, Pointers & Control	easy	2.0	1.5
How are elements of a one-dimensional array indexed in C?	Algorithmic Problem Solving & Arrays	easy	2.0	2.0
What does malloc return when the allocation fails?	Data Structures, Memory & Files	easy	2.0	0.5
What is the role of the main() function in a C program?	Fundamentals & System Architecture	easy	2.0	2.0
Which header file declares strlen, strcpy and strcmp?	Strings & Text Processing	easy	2.0	1.5
Insert a number at a given position in an array, shifting the elements after the position one position to the right.	Algorithmic Problem Solving & Arrays	medium	5.0	3.5
Explain how the compiler, assembler and linker cooperate to create an executable.	Fundamentals & System Architecture	medium	5.0	0.0
Write a program to find the largest number in an array.	Algorithmic Problem Solving & Arrays	medium	5.0	1.5

6.7 Knowledge Tracing with EKT

EKT [2] considers both the learner's response and the content of each exercise. At step t the learner answers exercise t on concept c_t with correctness r_t ($1 = \text{attained}$, $0 = \text{not attained}$). The implementation computes:

$$\begin{aligned}
x_e &= \text{embedding}(\text{question text}) \text{ (384 values, all-MiniLM-L6-v2)} \\
e_t &= \tanh(W_e \cdot x_e + E_c[c_t]) \text{ (exercise vector, 100 values)} \\
u_t &= [e_t, 0] \text{ if } r_t = 1, u_t = [0, e_t] \text{ if } r_t = 0 \\
h_t &= \text{LSTM}(u_t, h_{t-1}) \text{ (knowledge state, 100 units)} \\
y_t &= \text{sigmoid}(W \cdot h_t + b) \text{ (mastery of every concept)}
\end{aligned}$$

The next answer is predicted from the mastery of the next concept plus an exercise-aware term. With attention (the EKTA variant of [2]), the knowledge state used for the prediction is a weighted sum of past states, weighted by the similarity between the next exercise and each past exercise:

$$\begin{aligned}
\alpha_{t,j} &= \text{softmax}_j(\cos(e_{t+1}, e_j)), j \leq t; h'_t = \sum_j \alpha_{t,j} \cdot h_j \\
P(r_{t+1} = 1) &= \text{sigmoid}(y_t[c_{t+1}] + \text{MLP}([h'_t, e_{t+1}]))
\end{aligned}$$

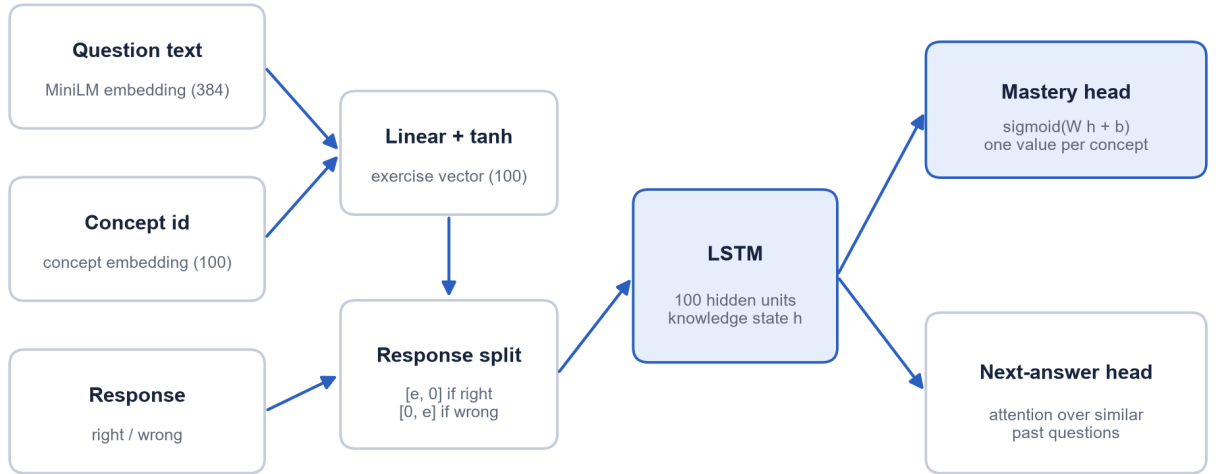


Figure 6.4: EKT model architecture as implemented

The recurrent part is a Long Short-Term Memory network [17]. The model is trained with binary cross-entropy on the next answer. An additional (auxiliary) loss trains the mastery output y_t directly on the next answer of the same concept. Without it, the reported mastery stayed close to 0.5 for every concept; with it, the mastery values are calibrated and rise when a learner answers correctly. Hyperparameters follow the paper: embedding 100, LSTM 100, dropout 0.3, Adam [18] with learning rate 0.001, batch size 32, 12 epochs and a 75/25 split of learners into training and test sets.

DKT baseline. DKT [3] receives only a one-hot vector of (concept, correctness), passes it through an LSTM of the same size and outputs the mastery of each concept. It never sees the question text, which makes it a direct test of the value of exercise awareness.

kt.py - EKT.forward()

```

def forward(self, concepts, ex_emb, correct):
    e = self.encode(concepts, ex_emb)                # (B, T, E)
    c = correct.float().unsqueeze(-1)
    x = torch.cat([e * c, e * (1 - c)], dim=-1)      # response-aware exercise input
    h, _ = self.lstm(x)
    h = self.drop(h)
    mastery = self.mastery(h)                        # (B, T, K)
    e_next, h_prev = e[:, 1:], h[:, :-1]
    if self.use_attention:
        # alpha_{t,j} ~ cos(e_{t+1}, e_j) for j <= t
        en = nn.functional.normalize(e, dim=-1)
        sim = en[:, 1:] @ en[:, :-1].transpose(1, 2) # (B, T-1, T-1)
        t = sim.size(1)
        causal = torch.tril(torch.ones(t, t, dtype=torch.bool, device=sim.device))
        att = torch.softmax(sim.masked_fill(~causal, -1e9) * 5.0, dim=-1)
        h_prev = att @ h[:, :-1]
    nxt = concepts[:, 1:].unsqueeze(-1)
    logits = mastery[:, :-1].gather(-1, nxt).squeeze(-1) + self.pred(torch.cat([h_prev, e_next],
    -1)).squeeze(-1)
    return logits, mastery

```

6.8 Knowledge Gap Identification

The OBE threshold and target define three levels and the gap rule:

Table 6.5: Classification rules

Mastery	Level	Knowledge gap?
below 50% (threshold)	Beginner	Yes
50% to below 70% (target)	Intermediate	Yes
70% and above	Expert	No

Table 6.6: Gap analysis of learner CSE230001 before remediation (round 1)

Concept	EKT mastery	Level	Gap
Algorithmic Problem Solving & Arrays	55%	Intermediate	Yes
Data Structures, Memory & Files	43%	Beginner	Yes
Fundamentals & System Architecture	31%	Beginner	Yes
Operators, Pointers & Control	43%	Beginner	Yes
Strings & Text Processing	35%	Beginner	Yes

All five concepts of this learner are below the 70% target and are therefore knowledge gaps. The decision uses the EKT mastery rather than the raw exam score: for example, the learner scored 75% on the single question about operators, but one answer is weak evidence, so EKT keeps the mastery near the class average until more answers are available.

6.9 Personalized Remedial Recommendation

For every weak concept the recommender retrieves the concept's curriculum context and the questions the learner attempted in the latest assessment, and sends the following prompt (adapted from the paper) to the primary model:

Generate a structured remedial recommendation for the concept "<concept>" for a learner who demonstrates <mastery>% concept mastery. The response should include five sections: Learning Objectives, Recommended Topics to Revise, Remedial Explanation, Two Practice Activities and Concept Gap Rationale, which will describe the specific weaknesses detected by the knowledge-tracing model and the reasoning behind the generated remedial content.

- **Learning objectives:** what the learner should be able to do after studying.
- **Topics to revise:** foundational topics with key points.
- **Remedial explanation:** the concept explained again from the curriculum.
- **Practice activities:** two hands-on exercises.
- **Concept gap rationale:** which gaps were detected and why this plan was generated (transparency).

The recommendation is scored by Gemini and DeepSeek (1 to 5). If the average score is below 3, it is regenerated with the reviewers' feedback, up to two more times. In later rounds the prompt asks for a more detailed explanation, simpler examples and additional practice.

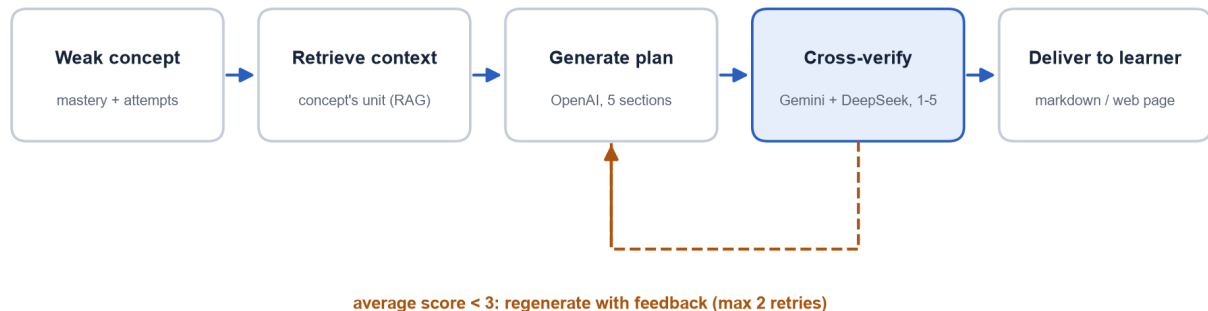


Figure 6.5: Remedial recommendation with cross-verification

6.10 Adaptive Evaluation and Re-testing

The adaptive test covers only the learner's weak concepts. For each concept the learner starts at Easy; a level is passed when the percentage of correct answers reaches the instructor's rule, and only then is the next level shown. The rule is written as "N and P": N questions per level and P percent to pass. A core course may use "5 and 80" (4 of 5 correct), an optional course "1 and 100". Questions already shown to the learner are excluded, so every round uses new questions and measures understanding rather than memory.

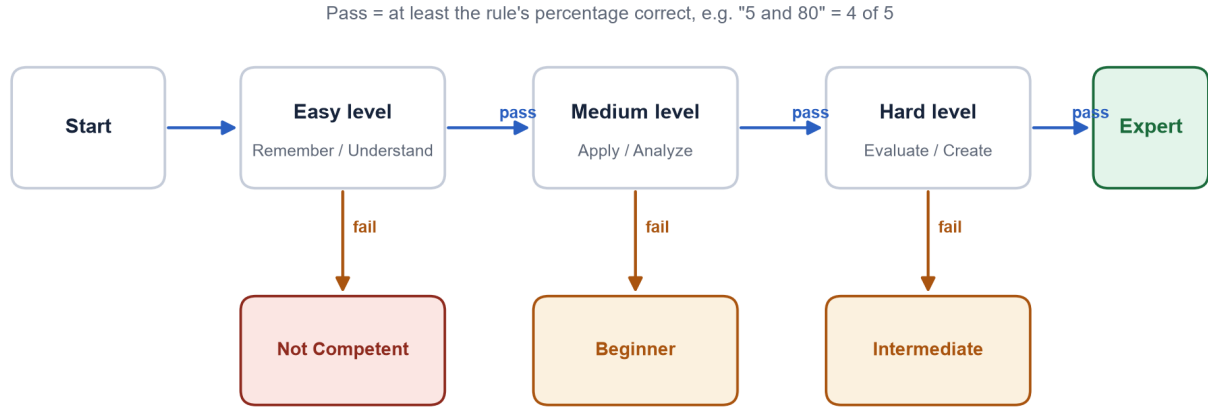


Figure 6.6: Adaptive evaluation and resulting categories

6.11 Continuous Learning Loop

The answers of the adaptive test are converted to OBE records (1 mark for a correct answer, 0 otherwise), appended to the learner's history and traced again by EKT. The loop stops when every concept reaches the target or after the maximum number of rounds (three by default).

pipeline.py - run_cycle()

```

def run_cycle(self, learner_id: str, records: list[Record], answer_fn: AnswerFn,
               study_fn: StudyFn | None = None, rule: EvaluationRule | None = None,
               max_rounds: int = 3, save: bool = True, seed: int = 0) -> list[RoundResult]:
    """Continuous learning loop: analyse -> recommend -> (study) -> evaluate -> re-trace."""
    history = list(records)
    seen: set[str] = set()
    rounds: list[RoundResult] = []
    for rnd in range(1, max_rounds + 1):
        gaps = self.analyze(history)
        state = {g.concept: round(g.mastery, 4) for g in gaps}
        weak = [g for g in gaps if g.is_gap]
        result = RoundResult(rnd, state, [g.concept for g in weak])
        rounds.append(result)
        if not weak:
            log.info("Round %d: no knowledge gaps left - target reached.", rnd)
            break
        result.recommendations = self.recommend(weak, rnd)
        if study_fn:
            study_fn(result.recommendations)
        session = self.evaluation(learner_id, [g.concept for g in weak], rule, seen, seed + rnd)
        session.run(answer_fn)
        seen |= session.seen
        result.evaluation = session.report()
        for r in session.records: # feed evaluation interactions back into knowledge tracing
            r.order = len(history)
            r.exam = f"SelfAssessment-R{rnd}"
            history.append(r)
    else:
        gaps = self.analyze(history)
        rounds.append(RoundResult(max_rounds + 1, {g.concept: round(g.mastery, 4) for g in gaps},
                                  [g.concept for g in gaps if g.is_gap]))
    if save:
        self.save_rounds(learner_id, rounds)
    return rounds

```

6.12 Evaluation Metrics

Table 6.7: Evaluation metrics

Metric	Definition	Used for
AUC	Probability that a randomly chosen correct answer receives a higher predicted probability than a randomly chosen wrong answer (Mann-Whitney U) [26]	Knowledge tracing
Accuracy	$(TP + TN) / \text{all predictions}$, threshold 0.5	Knowledge tracing
Precision	$TP / (TP + FP)$	Knowledge tracing
Recall	$TP / (TP + FN)$	Knowledge tracing
F1-score	$2 \times \text{precision} \times \text{recall} / (\text{precision} + \text{recall})$	Knowledge tracing
MAE	Mean of $ \text{actual} - \text{predicted probability} $	Knowledge tracing
BCE loss	$-\text{mean}[y \log p + (1 - y) \log(1 - p)]$	Training objective
Semantic similarity	Mean best S-BERT cosine similarity between generated and teacher questions	Question generation
Readability	Flesch-Kincaid grade = $0.39 (\text{words/sentences}) + 11.8 (\text{syllables/words}) - 15.59$ [19]	Question generation
Diversity	Self-BLEU / 100 [20], [21]; lower = less repetition	Question generation
Relevance (1-5)	Similarity mapped to 5 (> 0.9), 4 (> 0.8), 3 (> 0.7), 2 (≥ 0.6), 1 (< 0.6)	Question generation
Duplicate rate	Duplicates removed / questions generated	Question generation
Paired t-test	$t = \text{mean}(\text{pre} - \text{post}) / (\text{sd} / \sqrt{n})$, two-sided p-value	Before/after analysis
Improvement %	$(\text{post} - \text{pre}) / \text{pre} \times 100$	Before/after analysis

CHAPTER 7

Implementation

7.1 Development Environment

The system was developed in Python 3.13 on Windows 11 using Visual Studio Code, with a virtual environment (.venv) and the packages listed in requirements.txt. The web frontend is plain HTML, CSS and JavaScript served by FastAPI [30], with no build step.

7.2 Project Structure

Directory structure

```
AI-Personalized-Learning/
|-- config.yaml          all parameters (paper values)
|-- .env.example         template for API keys (real keys go in .env, ignored by git)
|-- requirements.txt     dependencies
|-- README.md           user guide
|-- data/
|   |-- sample/         curriculum_c_programming.txt, historical.csv, current_batch.csv
|   |-- store/          vector DB, concepts.json, question_bank.json, learner reports
|   (generated)
|   |-- models/         ekt.pt, dkt.pt, kt_history.csv, experiments.json (generated)
|-- plrs/               the Python package
|   |-- config.py       configuration loading
|   |-- data.py         OBE records, CSV I/O, synthetic data generator
|   |-- embeddings.py   MiniLM embeddings (offline hashing fallback)
|   |-- llm.py          OpenAI, Gemini, DeepSeek providers and offline mock
|   |-- rag.py          PDF loading, chunking, vector store, concept extraction
|   |-- kt.py           EKT and DKT models, training, metrics, knowledge state
|   |-- gap.py          OBE levels, gap analysis, gap matrix
|   |-- remedial.py     five-part recommendations with cross-verification
|   |-- qgen.py         MCQ generation, de-duplication, question bank
|   |-- evaluation.py   adaptive evaluation sessions
|   |-- pipeline.py     continuous learning loop (Algorithm 1)
|   |-- metrics.py      similarity, readability, Self-BLEU, t-test
|   |-- experiments.py  reproduction of the paper's evaluation
|   |-- cli.py, __main__.py command-line interface
|   |-- api.py          FastAPI REST API
|   |-- static/index.html web frontend
|-- tests/              30 pytest test cases
```

7.3 Implementation of the Modules

7.3.1 LLM providers and offline fallback

All generative calls go through one interface, `generate(prompt, task, payload)`. Real providers send the prompt to their API [32]: OpenAI and DeepSeek (OpenAI-compatible endpoint) through the `openai` client, Gemini through `google-generativeai`. The answer is parsed as JSON, tolerating code fences and extra text. If a provider has no key, its package is missing or a call fails, a deterministic offline mock produces an answer of the same JSON shape from the payload (for example, questions built from curriculum sentences with corrupted statements as distractors). After an account-level error (no credit, invalid key, unknown model) the provider is switched off for the rest of the run, so the system does not wait for a failing network call on every request.

7.3.2 Knowledge tracing

`kt.py` implements the EKT and DKT models in PyTorch [24], an encoder that maps concepts to indices and caches the embedding of each question text, batching with padding and masks, training with gradient clipping, evaluation with loss, AUC, MAE, accuracy, precision, recall and F1, and saving/loading of checkpoints. The `KnowledgeTracer` class returns the knowledge state of a learner after every interaction.

7.3.3 Gap analysis, recommendations, questions and evaluation

`gap.py` computes OBE concept scores, levels, gap flags and the class-wide gap matrix. `remedial.py` builds the prompt, calls the primary model, cross-verifies and renders a Markdown version of the plan (as in Figure 2 of the paper). `qgen.py` implements the generation, refinement, de-duplication and verification pipeline and the persistent question memory. `evaluation.py` implements the adaptive session with rule parsing, grading, level progression and conversion of answers to OBE records.

gap.py - analyze_learner()

```
def analyze_learner(records: list[Record], tracer=None, concepts: list[str] | None = None) ->
    list[ConceptGap]:
    _, target = obe_levels()
    scores = concept_scores(records)
    mastery = tracer.knowledge_state(records) if tracer is not None else {}
    names = concepts or list(dict.fromkeys(list(mastery) + list(scores)))
    out = []
    for c in names:
        m = mastery.get(c, scores.get(c, 50.0) / 100.0)
        mine = [r for r in records if r.concept == c]
        recent = [r for r in mine if r.exam == mine[-1].exam][-10:] if mine else [] # latest
            exam/session
        attempts = [{"question": r.question_text, "difficulty": r.difficulty, "bloom_level":
            r.bloom_level,
            "correct": bool(r.attained())} for r in recent]
        out.append(ConceptGap(c, scores.get(c), m, classify(100 * m), 100 * m < target, attempts))
    return out
```

7.4 Command-Line Interface

Table 7.1: Commands of the command-line interface (python -m plrs <command>)

Command	Function
sample-data	Generate synthetic OBE data
ingest <file>	Index a curriculum and extract concepts
concepts	List the concepts
query <text>	Semantic search over the curriculum
train-kt --model ekt dkt all	Train knowledge-tracing models
analyze --learner <id>	Knowledge-gap analysis for a learner
gap-matrix	Class-wide gap matrix
recommend --learner <id>	Remedial plan for a learner
generate-mcq	Generate and verify MCQs
evaluate --rule "5 and 80"	Take an adaptive test in the terminal
cycle --learner <id>	Full learning loop with a simulated learner
experiments	Reproduce the evaluation tables
demo	Run everything end to end
serve	Start the web frontend and REST API

7.5 User Interface

The web frontend guides a learner or teacher through one complete loop on a single page. The screenshots below were captured from the running application.

Personalized Learning Assistant

Generator: mock:openai · Verifiers: mock:gemini, mock:deepseek · Knowledge tracing: EKT

1
Learner exam results

Load a sample learner, upload an OBE results CSV, or type the marks in yourself. You can edit any row before analysing.

Sample learner
Upload CSV
Enter manually

CSE230001
Load

Learner ID
me

CONCEPT
QUESTION
DIFFICULTY
MAX
MARKS

No exam results yet.

+ Add row

Analyse & get recommendations

Figure 7.1: Home page: loading a sample learner, uploading a CSV or entering results manually

1
Learner exam results

Load a sample learner, upload an OBE results CSV, or type the marks in yourself. You can edit any row before analysing.

Sample learner
Upload CSV
Enter manually

CSE230001
Load

Learner ID
CSE230001

CONCEPT	QUESTION	DIFFICULTY	MAX	MARKS	
Data Structures, Memory Management, and File Handling	Which function opens a file f	easy	2	0	×
Operators, Pointers and Control Structures	Which operator is used to ol	easy	2	1.5	×
Algorithmic Problem Solving and Array Manipulation	How are elements of a one-α	easy	2	2	×
Data Structures, Memory Management, and File Handling	What does malloc return wh	easy	2	0.5	×
Fundamentals and System Architecture	What is the role of the main(	easy	2	2	×
String Manipulation and Text Processing	Which header file declares st	easy	2	1.5	×
Algorithmic Problem Solving and Array Manipulation	Insert a number at a given p	medium	5	3.5	×
Fundamentals and System Architecture	Explain how the compiler, as	medium	5	0	×
Algorithmic Problem Solving and Array Manipulation	Write a program to find the	medium	5	1.5	×

+ Add row

Analyse & get recommendations

Figure 7.2: Exam results of learner CSE230001 loaded and editable

33

1 Learner exam results

Load a sample learner, upload an OBE results CSV, or type the marks in yourself. You can edit any row before analysing.

Sample learner [Upload CSV](#) Enter manually

No file chosen Columns: learner_id, question_id, question_text, concept, difficulty, max_marks, marks

Learner ID

CONCEPT	QUESTION	DIFFICULTY	MAX	MARKS	
Data Structures, Memory Management, and File Handling	Which function opens a file t	easy	2	0	×
Operators, Pointers and Control Structures	Which operator is used to ol	easy	2	1.5	×
Algorithmic Problem Solving and Array Manipulation	How are elements of a one-d	easy	2	2	×
Data Structures, Memory Management, and File Handling	What does malloc return wh	easy	2	0.5	×
Fundamentals and System Architecture	What is the role of the main	easy	2	2	×
String Manipulation and Text Processing	Which header file declares st	easy	2	1.5	×
Algorithmic Problem Solving and Array Manipulation	Insert a number at a given p	medium	5	3.5	×
Fundamentals and System Architecture	Explain how the compiler, as	medium	5	0	×
Algorithmic Problem Solving and Array Manipulation	Write a program to find the	medium	5	1.5	×

Figure 7.3: Uploading an OBE results CSV

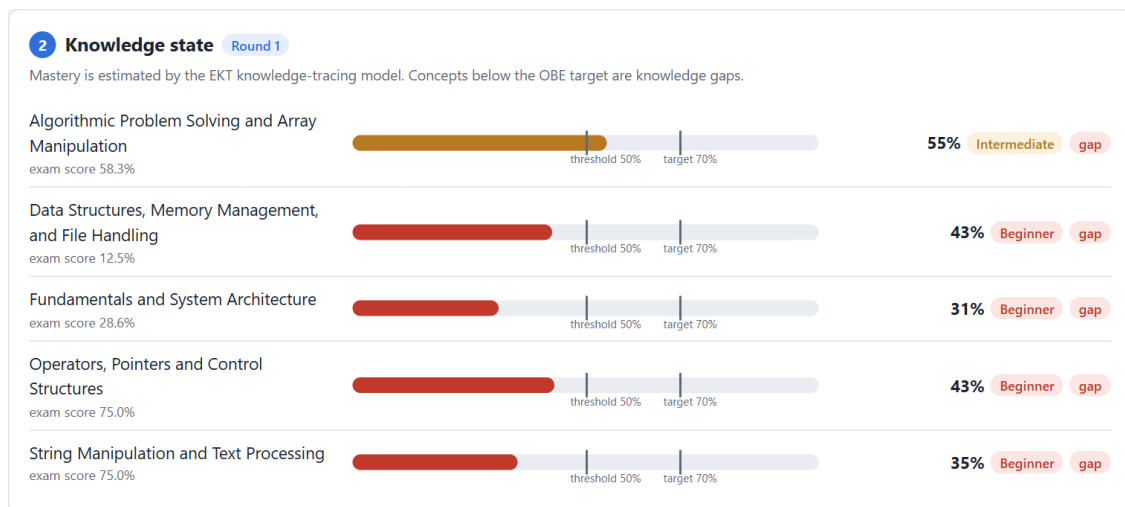


Figure 7.4: Knowledge state: EKT mastery per concept with threshold and target

Algorithmic Problem Solving and Array Manipulation
current mastery 55%

1. Learning objectives

- Explain the key ideas of Algorithmic Problem Solving and Array Manipulation.
- Apply element correctly in small programs.
- Identify common mistakes involving elements.

2. Topics to revise

- 1. element**
 - An array stores a fixed number of elements of the same type in contiguous memory locations.
 - Array elements in C are indexed from 0, so the last element of an array of size n has index $n - 1$.
- 2. elements**
 - Linear search checks each element in turn and takes time proportional to the number of elements.
 - Inserting an element at a given position requires shifting all later elements one position to the right.
- 3. array**
 - Accessing an array outside its bounds is not checked by C and leads to undefined behaviour.
 - Traversal visits every element of an array using a loop over all valid indices.
- 4. largest**
 - To find the largest element, assume the first element is the largest and compare it with every other element, updating the assumption when a bigger value is found.

3. Explanation

Problem solving begins by decomposing a problem into smaller steps before writing any code. An array stores a fixed number of elements of the same type in contiguous memory locations. Array elements in C are indexed from 0, so the last element of an array of size n has index $n - 1$. Accessing an array outside its bounds is not checked by C and leads to undefined behaviour.

4. Practice activities

- **Practice 1: element:** Write and test a short program that illustrates this rule: An array stores a fixed number of elements of the same type in contiguous memory locations.
Input: at least two test cases of your choice → Expected: output that matches your hand-traced result
- **Practice 2: elements:** Write and test a short program that illustrates this rule: Linear search checks each element in turn and takes time proportional to the number of elements.
Input: at least two test cases of your choice → Expected: output that matches your hand-traced result

5. Why this was recommended

The knowledge-tracing model estimates your mastery of 'Algorithmic Problem Solving and Array Manipulation' at 55%, below the OBE target. In your latest assessment you answered 2 of 3 questions on this concept correctly; for example you missed "Write a program to find the largest number in an array.". The plan focuses on the topics that concept's questions depend on.

Cross-verified by mock:gemini, mock:deepseek: 5.0/5

Figure 7.5: Personalized remedial recommendation with its five sections

Algorithmic Problem Solving and Array Manipulation easy · Understand

1. Which statement correctly describes Algorithmic Problem Solving and Array Manipulation?

- ☐ The top three elements of an array cannot be found in a single pass by maintaining three running maxima.
- ☐ Problem solving begins by decomposing a problem into smaller steps after writing any code.
- ☐ The top three elements of an array can be found in a single pass by maintaining three running maxima.
- ☐ Standard input and output are not performed with the scanf and printf functions declared in stdio.h.

2. Which statement correctly describes Algorithmic Problem Solving and Array Manipulation?

- ☐ It is never necessary to consider this: linear search checks each element in turn and takes time proportional to the number of elements.
- ☐ It is never necessary to consider this: the functions in ctype.h such as toupper, tolower, isalpha and isdigit classify and convert individual characters.
- ☐ Linear search checks each element in turn and takes time proportional to the number of elements.
- ☐ It is never necessary to consider this: every C program starts executing from the main function, which returns an integer status to the operating system.

3. Fill in the blank (Algorithmic Problem Solving and Array Manipulation): "To find the largest element, assume the first element is the largest and compare it with every other element, updating the ____ when a bigger value is found."

- ☐ order
- ☐ largest
- ☐ search
- ☐ assumption

Submit answers

Figure 7.6: Adaptive self-assessment: generated questions at the Easy level

Algorithmic Problem Solving and Array Manipulation medium · Apply

▼ easy level (100%) — show answers

1. Which statement correctly describes Algorithmic Problem Solving and Array Manipulation?

- ☐ The top three elements of an array cannot be found in a single pass by maintaining three running maxima.
- ☐ Problem solving begins by decomposing a problem into smaller steps after writing any code.
- ☒ The top three elements of an array can be found in a single pass by maintaining three running maxima.
- ☐ Standard input and output are not performed with the scanf and printf functions declared in stdio.h.

Based on the course material: The top three elements of an array can be found in a single pass by maintaining three running maxima.

2. Which statement correctly describes Algorithmic Problem Solving and Array Manipulation?

- ☐ It is never necessary to consider this: linear search checks each element in turn and takes time proportional to the number of elements.
- ☐ It is never necessary to consider this: the functions in ctype.h such as toupper, tolower, isalpha and isdigit classify and convert individual characters.
- ☒ Linear search checks each element in turn and takes time proportional to the number of elements.
- ☐ It is never necessary to consider this: every C program starts executing from the main function, which returns an integer status to the operating system.

Based on the course material: Linear search checks each element in turn and takes time proportional to the number of elements.

3. Fill in the blank (Algorithmic Problem Solving and Array Manipulation): "To find the largest element, assume the first element is the largest and compare it with every other element, updating the ____ when a bigger value is found."

- ☐ order
- ☐ largest
- ☐ search
- ☒ assumption

Based on the course material: To find the largest element, assume the first element is the largest and compare it with every other element, updating the assumption when a bigger value is found.

easy: 100% passed — current level: Beginner

1. A learner must apply ideas from Algorithmic Problem Solving and Array Manipulation to solve a problem. Which statement should guide the solution?

- ☐ A two-dimensional array stores a matrix in row-major order in C.
- ☐ It is never necessary to consider this: every C program starts executing from the main function, which returns an integer status to the operating system.
- ☐ It is never necessary to consider this: a palindrome check compares characters from both ends of the string moving towards the middle.
- ☐ It is never necessary to consider this: a two-dimensional array stores a matrix in row-major order in C.

2. Which of the following statements about Algorithmic Problem Solving and Array Manipulation is INCORRECT?

- ☐ Accessing an array outside its bounds is not checked by C and leads to undefined behaviour.
- ☐ An array stores a fixed number of elements of the same type in contiguous memory locations.
- ☐ Problem solving begins by decomposing a problem into smaller steps after writing any code.
- ☐ Binary search repeatedly halves a sorted array and is much faster than linear search for large inputs.

3. A learner must apply ideas from Algorithmic Problem Solving and Array Manipulation to solve a problem. Which statement should guide the solution?

- ☐ To find the largest element, assume the last element is the largest and compare it with every other element, updating the assumption when a bigger value is found.
- ☐ To find the largest element, assume the first element is the largest and compare it with every other element, updating the assumption when a bigger value is found.
- ☐ The top three elements of an array cannot be found in a single pass by maintaining three running maxima.
- ☐ It is never necessary to consider this: inserting an element at a given position requires shifting all later elements one position to the right.

Submit answers

Figure 7.7: Graded level with correct and wrong answers and explanations

4 Self-assessment

After studying the recommendations, take an adaptive test on your weak concepts: Easy → Medium → Hard. You move up a level only when you pass the current one.

Rule *questions per level and pass %, e.g. 5 and 80 or 1 and 100 [Start self-assessment](#)

Algorithmic Problem Solving and Array Manipulation Expert

▼ easy level (100%) — show answers

1. Which statement correctly describes Algorithmic Problem Solving and Array Manipulation?

- ☐ The top three elements of an array cannot be found in a single pass by maintaining three running maxima.
- ☐ Problem solving begins by decomposing a problem into smaller steps after writing any code.
- ☒ The top three elements of an array can be found in a single pass by maintaining three running maxima.
- ☐ Standard input and output are not performed with the scanf and printf functions declared in stdio.h.

Based on the course material: The top three elements of an array can be found in a single pass by maintaining three running maxima.

2. Which statement correctly describes Algorithmic Problem Solving and Array Manipulation?

- ☐ It is never necessary to consider this: linear search checks each element in turn and takes time proportional to the number of elements.
- ☐ It is never necessary to consider this: the functions in ctype.h such as toupper, tolower, isalpha and isdigit classify and convert individual characters.
- ☒ Linear search checks each element in turn and takes time proportional to the number of elements.
- ☐ It is never necessary to consider this: every C program starts executing from the main function, which returns an integer status to the operating system.

Based on the course material: Linear search checks each element in turn and takes time proportional to the number of elements.

3. Fill in the blank (Algorithmic Problem Solving and Array Manipulation): "To find the largest element, assume the first element is the largest and compare it with every other element, updating the ____ when a bigger value is found."

- ☐ order
- ☐ largest
- ☐ search
- ☒ assumption

Based on the course material: To find the largest element, assume the first element is the largest and compare it with every other element, updating the assumption when a bigger value is found.

easy: 100% passed — current level: Beginner

► medium level (100%) — show answers

medium: 100% passed — current level: Intermediate

► hard level (100%) — show answers

hard: 100% passed — current level: Expert

Data Structures, Memory Management, and File Handling Not Competent

► easy level (67%) — show answers

easy: 67% not passed — current level: Not Competent

Fundamentals and System Architecture Not Competent

► easy level (67%) — show answers

easy: 67% not passed — current level: Not Competent

Operators, Pointers and Control Structures Not Competent

► easy level (67%) — show answers

easy: 67% not passed — current level: Not Competent

String Manipulation and Text Processing Beginner

► easy level (100%) — show answers

easy: 100% passed — current level: Beginner

► medium level (67%) — show answers

medium: 67% not passed — current level: Beginner

[Update my knowledge state → next round](#)

Your answers are added to your history and re-analysed.

Figure 7.8: Self-assessment completed for all weak concepts

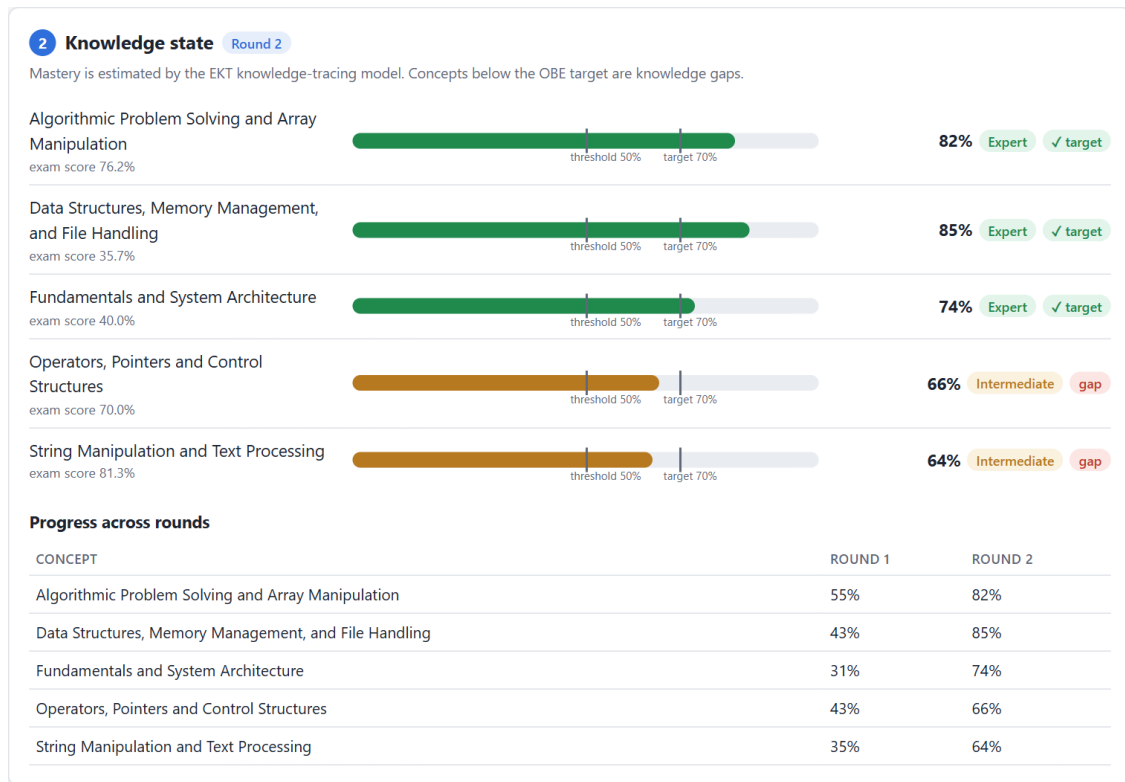


Figure 7.9: Round 2: updated knowledge state and progress across rounds

AI-Driven Personalized Learning & Remedial Recommendation 1.0.0 OAS 3.1

/openapi.json

default

GET	/sample-learners	Sample Learners	⌵
GET	/sample-learners/{learner_id}	Sample Learner	⌵
POST	/results/upload	Upload Results	⌵
GET	/health	Health	⌵
POST	/curriculum	Upload Curriculum	⌵
GET	/concepts	Concepts	⌵
POST	/analyze	Analyze	⌵
POST	/recommend	Recommend	⌵
POST	/mcq	Generate Mcq	⌵
POST	/evaluation	Start Evaluation	⌵
POST	/evaluation/{sid}/answer	Answer	⌵
GET	/evaluation/{sid}	Evaluation Report	⌵

Figure 7.10: Interactive REST API documentation (/docs)

7.6 Testing

The project has 30 automated test cases (pytest). The tests use a temporary data store, the offline embedder and the mock models, so they never call external APIs. All 30 tests pass.

Table 7.2: Test cases

Test	What is verified	Result
test_chunk_text_respects_size_and_overlap	Chunks respect size, start at sentences and overlap	Pass
test_split_sections_finds_units	Curriculum units are detected	Pass
test_ingest_and_concept_extraction	Five concepts with course outcomes are extracted	Pass
test_context_prefers_own_section	Retrieval stays inside the concept's unit	Pass
test_kb_roundtrip	Vector DB and concepts save and load	Pass
test_parse_json_variants	LLM answers with fences or extra text are parsed	Pass
test_mock_mcq_shape	Offline questions have 4 options and a valid key	Pass
test_fallback_switches_off_after_account_error	A provider with no credit is called only once	Pass
test_classify_thresholds	Beginner / Intermediate / Expert boundaries (49.9, 50, 69.9, 70)	Pass
test_concept_scores_and_gaps	Concept scores and gap flags	Pass
test_gap_matrix	Class-wide attainment percentages	Pass
test_sample_data_shape	Synthetic batch has 5 concepts and 16 questions	Pass
test_auc_score	AUC equals 1.0 for perfect and 0.5 for random ranking	Pass
test_classification_metrics	Accuracy, precision, recall, F1	Pass
test_tracer_knowledge_state	Mastery for every concept, within 0 and 1	Pass
test_tracer_save_load	Saved EKT model gives identical results	Pass
test_generate_dedup_and_verify	Generated questions are verified; a repeated batch is fully removed as duplicates	Pass
test_remedial_has_five_sections	Every recommendation has all five sections	Pass
test_rule_parse	"5 and 80" means 4 correct answers needed	Pass
test_adaptive_evaluation (2 cases)	All correct -> Expert; all wrong -> Not Competent	Pass
test_paired_t_test_matches_reference	t-test p-values match the textbook value (0.0734)	Pass
test_text_metrics	Readability, diversity and relevance behave correctly	Pass
test_run_cycle	The full learning loop runs and saves reports	Pass
test_health_and_concepts, test_frontend_and_sample_endpoints	API health, concepts and the web page	Pass
test_analyze_and_recommend	API analysis and recommendations	Pass
test_evaluation_flow	API evaluation session; answers are not leaked	Pass
test_results_upload	CSV upload, including a rejected invalid file	Pass
test_evaluation_records_feed_back	Evaluation answers can be re-analysed	Pass

7.7 Problems Encountered and Solutions

Several problems were found by inspecting real outputs during development. They are recorded here because each one changed the design.

Table 7.3: Problems found during development and how they were solved

Problem observed	Cause	Solution
Generated text started with broken words (e.g. "oops" from "loops")	Chunks were cut at a fixed character position	Cut chunks at sentence boundaries and start each overlap at a sentence start
Questions about strings used sentences about pointers	A 1000-character chunk could contain two units	Split the curriculum by unit before chunking; retrieve from the concept's own unit
EKT mastery was about 0.5 for every concept	The mastery output received too little training signal	Added an auxiliary loss on the mastery output
Mastery fell after nine correct answers, even for DKT	The simulator ordered questions easy to hard, so models learned position patterns	Shuffle question order in simulated exams
Unrelated questions appeared in the bank	The experiment and the demo wrote to the same question file at the same time	Give experiments their own isolated question memory
Very slow runs when API keys had no credit	Every call waited for a failing network request	Switch a provider off after an account-level error
Gemini returned "model not found"	Google retired gemini-1.5-flash	Updated the model name in config.yaml
The first request of the web app was slow	Loading the embedding model takes about a minute	Load all models once at server start-up

CHAPTER 8

Results and Evaluation

8.1 Experimental Setup

The experiments use the sample course "Procedural Programming using C" with five units. Because institutional data was not available, learner data was generated by a simulator based on Item Response Theory [23]: each simulated learner has an ability per concept, and the probability of a correct answer is

$$P(\text{correct}) = 1 / (1 + \exp(-1.7 (\theta_{\text{concept}} - b_{\text{difficulty}}))), \quad b = -1 \text{ (easy)}, 0 \text{ (medium)}, +1 \text{ (hard)}$$

with a small practice effect after each answer. The historical dataset trains the knowledge-tracing models; the current batch has the same shape as the paper's recommendation dataset.

Table 8.1: Datasets used in this project

Item	Historical data (KT training)	Current batch (analysis)
Concepts	5	5
Learners	600	262
Unique Questions	30	16
# Records	24000	2358
Avg. questions attempted per learner	40.0	9.0
Avg. concepts attempted per learner	5.0	4.75
Avg. attainment percentage for learner	58.02	78.41

For comparison only: the paper [1] used real OBE data from the AMPLE LMS with 382 concepts, 1,276 learners and 359,681 learner-concept interactions over a 70+ course curriculum. That dataset was not available to this project and none of its values are used as our results.

Table 8.2: Experimental environment

Item	Value
Hardware	Intel Core i5-1335U (10 cores), 15.7 GB RAM, no GPU
Operating system	Windows 11 Pro
Python / PyTorch	3.13.2 / 2.14 (CPU)
Embedding model	all-MiniLM-L6-v2 (sentence-transformers)
Generative models	Offline mock for all reported runs (see Section 8.3)
KT split / epochs / seed	75% / 25% of learners, 12 epochs, seed 42
Remediation study	30 simulated learners, rule "3 and 67"

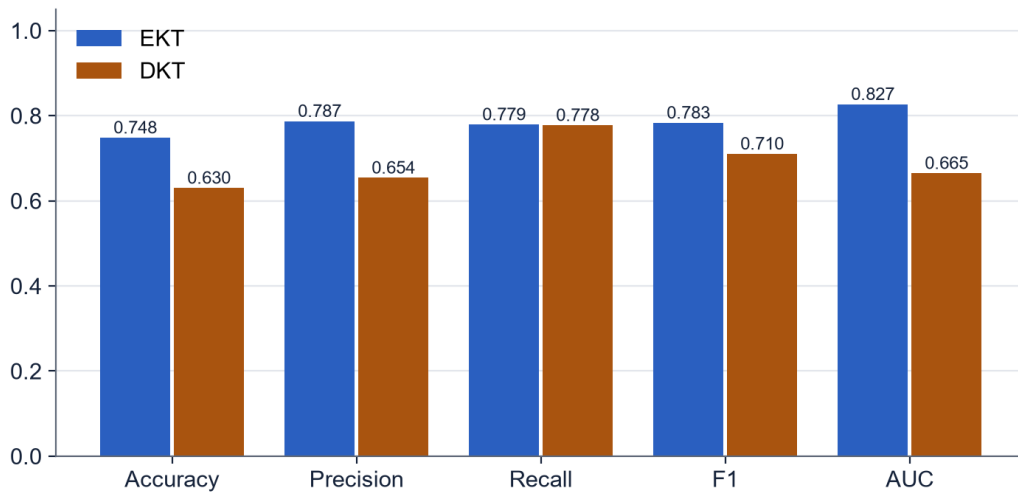
8.2 Knowledge Tracing Results

Table 8.3: Knowledge tracing performance on the test set (after 12 epochs)

Model	AUC	Accuracy	Precision	Recall	F1	MAE	Loss
EKT	0.827	0.748	0.787	0.779	0.783	0.331	0.504
DKT	0.665	0.630	0.654	0.778	0.710	0.447	0.637

Table 8.4: Knowledge tracing performance on the training set

Model	AUC	Accuracy	F1	MAE	Loss
EKT	0.848	0.770	0.801	0.318	0.476
DKT	0.700	0.656	0.728	0.436	0.620

**Figure 8.1: Test-set metrics of EKT and DKT**

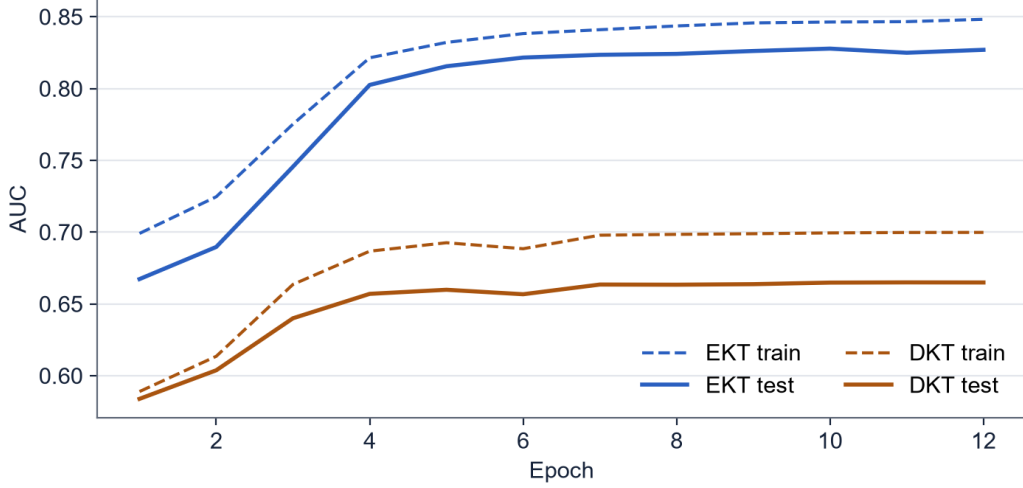


Figure 8.2: AUC over training epochs

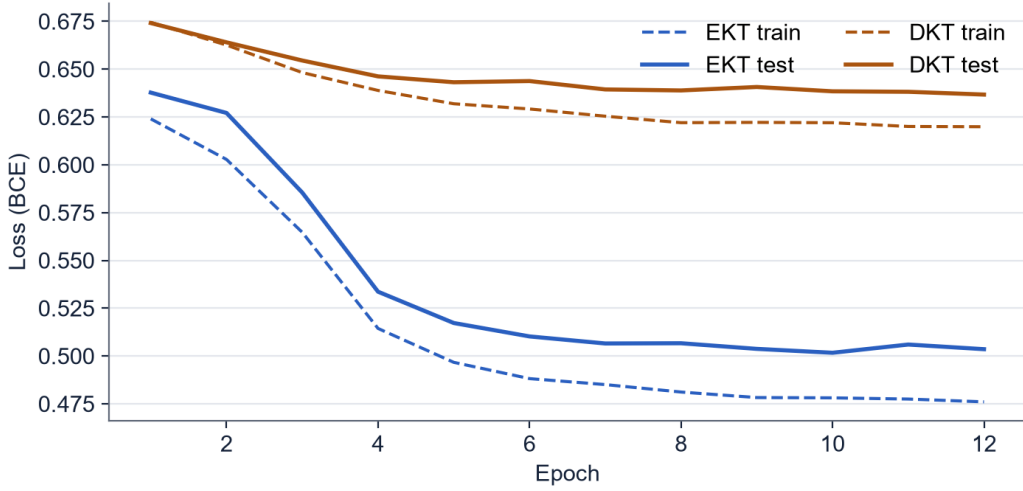


Figure 8.3: Loss over training epochs

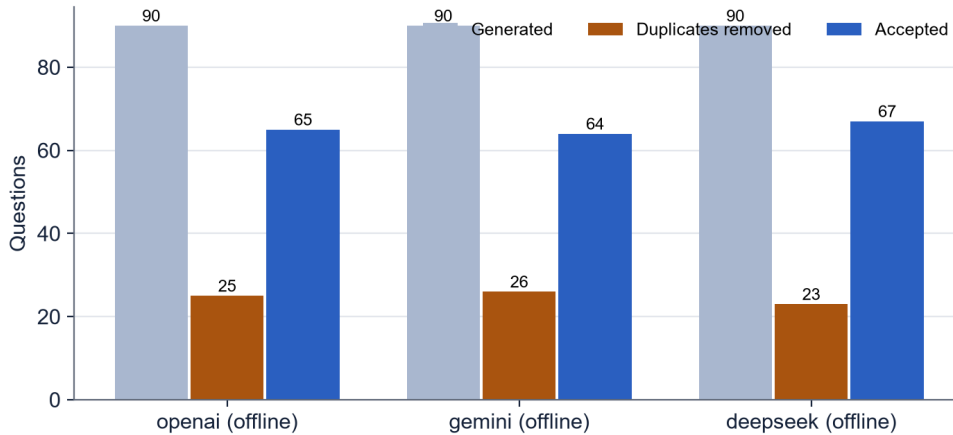
EKT outperforms DKT on every metric: test AUC 0.827 against 0.665, and test F1 0.783 against 0.710. The only difference between the models is that EKT reads the text of each question, which confirms the value of exercise awareness reported in [2] and [1]. The small gap between training and test AUC shows that the model is not overfitting after 12 epochs.

8.3 Question Generation Results

All question-generation runs reported here used the offline mock generator. During this project the available OpenAI and DeepSeek accounts had no remaining credit and the configured Gemini model had been retired, so the real models could not be evaluated. The mock builds questions from curriculum sentences; its purpose is to test the pipeline (retrieval, de-duplication, verification, storage), not to measure question quality.

Table 8.5: Question generation pipeline statistics and quality metrics

Generator	Generated	Duplicates	Accepted	Duplicate rate	Similarity	Readability (FK)	Diversity	Relevance
openai (offline)	90	25	65	27.8%	0.429	13.7	0.922	1.08
gemini (offline)	90	26	64	28.9%	0.436	13.5	0.924	1.11
deepseek (offline)	90	23	67	25.6%	0.426	13.5	0.926	1.04

**Figure 8.4: Generated, duplicate and accepted questions**

The pipeline behaves as designed: about a quarter of the generated questions were detected as near-duplicates (cosine similarity above 0.85) and removed, and every stored question has four options, a valid key, a difficulty and a Bloom's level. The quality metrics are low (relevance about 1 out of 5), which is expected because the mock copies curriculum sentences instead of writing teacher-style questions. For comparison, the paper reports a semantic similarity of 0.5642, a readability of 10.53, a diversity of 0.12 and a relevance of 1.56 with real models [1]. Measuring this properly requires real API access.

8.4 Knowledge Gap Analysis of the Class

Table 8.6: Knowledge gap matrix of the current batch (target 70%)

Concept	Attained	Not attained	Learners
Algorithmic Problem Solving & Arrays	49.8%	50.2%	257
Data Structures, Memory & Files	62.8%	37.2%	250
Fundamentals & System Architecture	63.1%	36.9%	244
Operators, Pointers & Control	73.0%	27.0%	248
Strings & Text Processing	60.6%	39.4%	246

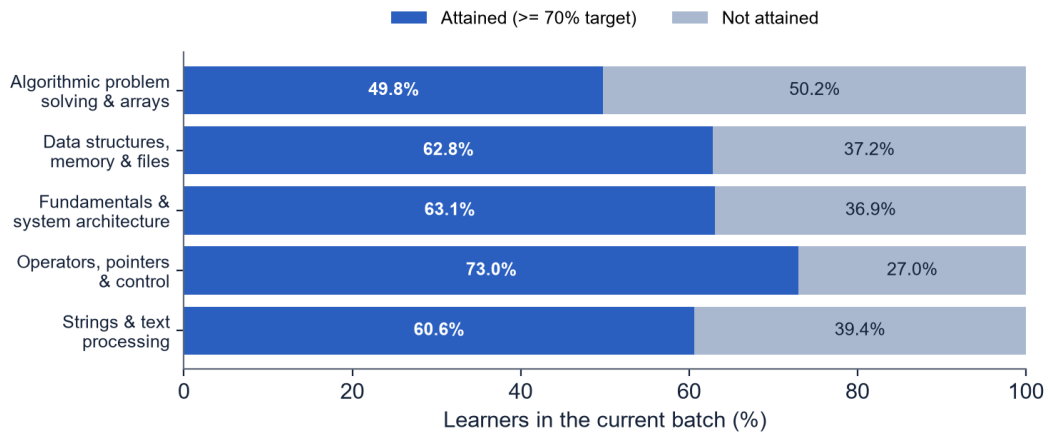


Figure 8.5: Share of learners attaining each concept

8.5 Case Study: Learning Loop of One Learner

Table 8.7: EKT mastery of learner CSE230001 before and after one round of remediation

Concept	Round 1	Round 2	Change	Reached target?
Algorithmic Problem Solving & Arrays	55%	85%	+31 points	Yes
Data Structures, Memory & Files	43%	90%	+47 points	Yes
Fundamentals & System Architecture	31%	79%	+48 points	Yes
Operators, Pointers & Control	43%	75%	+32 points	Yes
Strings & Text Processing	35%	74%	+38 points	Yes

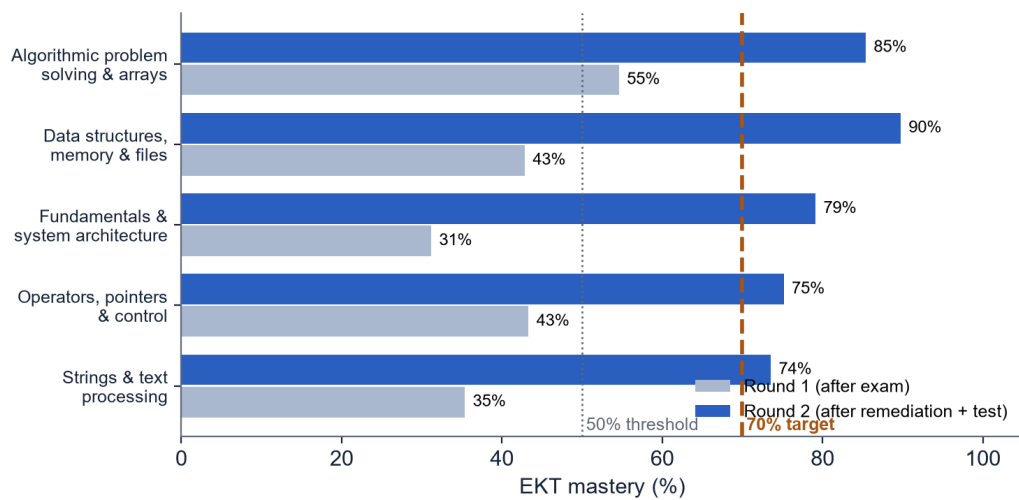


Figure 8.6: Mastery of learner CSE230001 in rounds 1 and 2

In round 1 all five concepts were knowledge gaps. The learner received five recommendations and took the adaptive test; the answers were fed back into EKT. In round 2 every concept was at or above the 70% target, so the loop stopped. The test answers and the learning effect of studying were simulated in this run.

8.6 Before vs After Analysis

Table 8.8: Paired t-test of scores before and after remediation (simulated learners)

Concept	Pre-test	Post-test	Difference	Improvement %	t	p-value	Significant
Algorithmic Problem Solving & Arrays	79.4	87.5	+8.1	10.1%	-1.95	0.0604	No
Data Structures, Memory & Files	67.8	88.1	+20.3	29.9%	-5.36	< 0.0001	Yes
Fundamentals & System Architecture	71.1	86.2	+15.1	21.2%	-3.54	0.0014	Yes
Operators, Pointers & Control	72.8	78.6	+5.8	8.0%	-1.22	0.2338	No
Strings & Text Processing	66.7	92.5	+25.8	38.7%	-6.66	< 0.0001	Yes

$$\text{Improvement \%} = (\text{Post-test} - \text{Pre-test}) / \text{Pre-test} \times 100$$

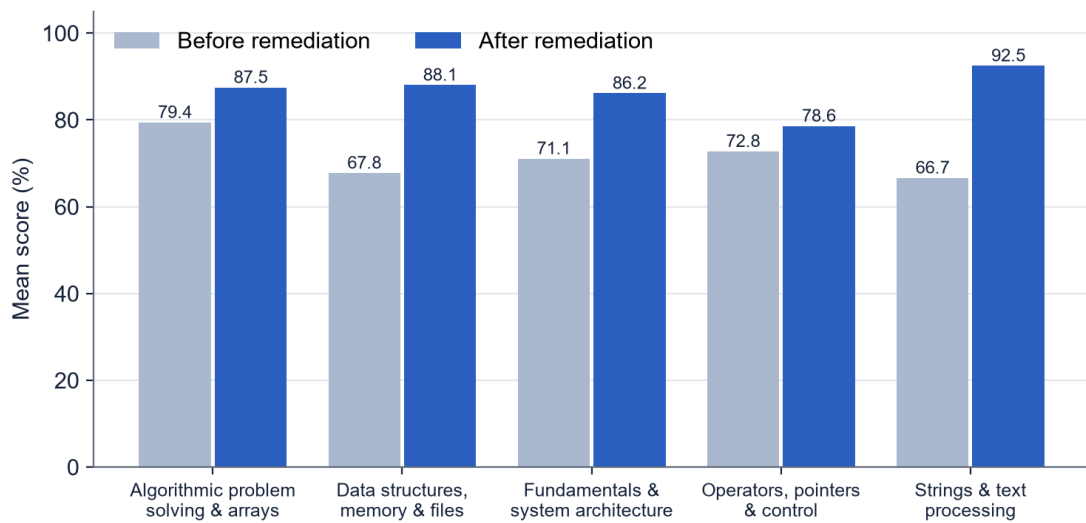


Figure 8.7: Mean score per concept before and after remediation

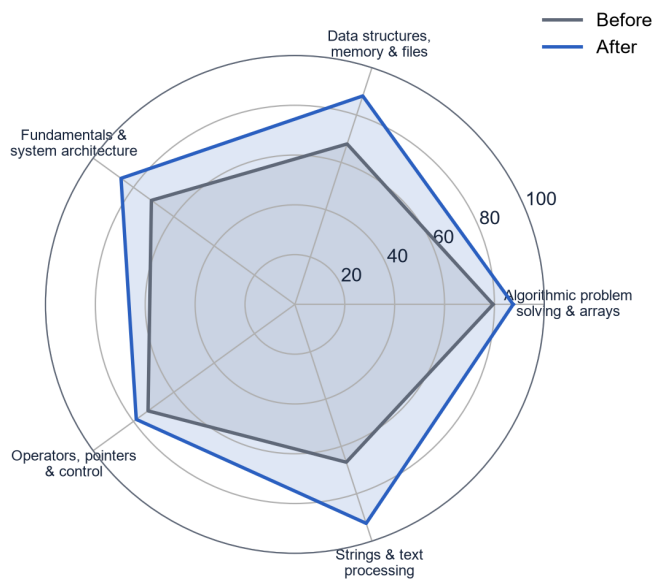


Figure 8.8: Concept-level comparison before and after remediation

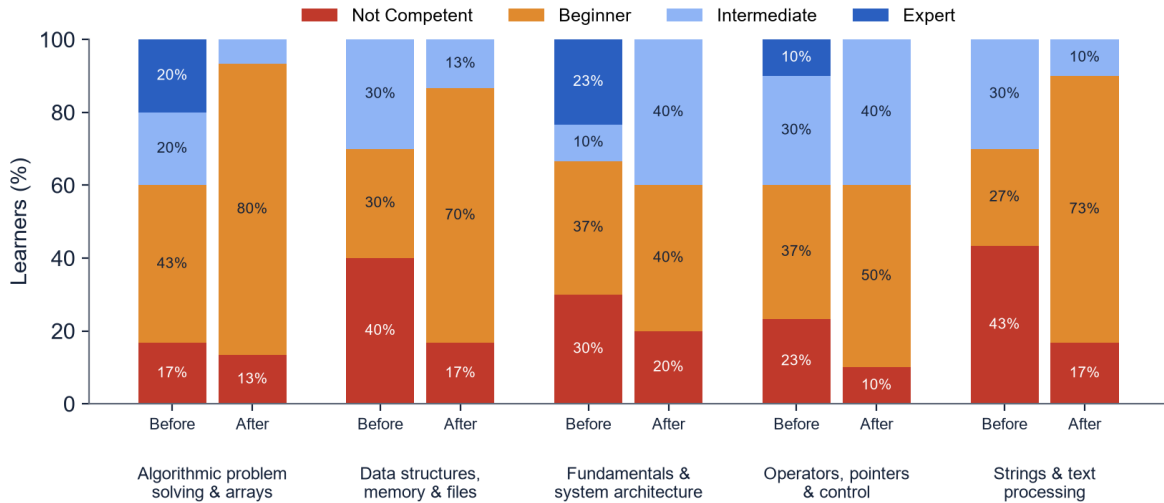


Figure 8.9: Distribution of competency levels before and after remediation

Mean scores increased on all five concepts. The increase was statistically significant ($p < 0.05$) for three concepts and not significant for two, whose pre-test scores were already high, leaving less room for improvement with only 30 learners. The negative t-values follow the paper's convention (pre minus post). The paper's real classroom study found significant improvements ($p < 0.0001$) on all five concepts [1].

These learners and their learning gain are simulated: after a recommendation, the ability of a simulated learner on that concept is increased. The analysis therefore demonstrates that the loop, the adaptive test and the statistics work correctly; it is not evidence of a real classroom effect.

8.7 User Evaluation

The paper evaluated learner satisfaction with a survey (its Table 7 and Figure 9). No survey with real learners was conducted in this project, so no ratings are reported. The questionnaire is provided in Appendix H so that it can be used in a future evaluation with real students.

8.8 Comparison with the Results Reported in the Paper

Table 8.9: Our results compared with the paper's reported results

Measure	Paper [1]	This project	Note
EKT test AUC	0.9334	0.827	Different data: real vs synthetic
DKT test AUC	0.9311	0.665	Same ranking: EKT > DKT
Learners / concepts (KT data)	1,276 / 382	600 / 5	Synthetic, one course
MCQ semantic similarity	0.5642	0.429	Ours: offline mock
Pre/post significance	$p < 0.0001$ on all 5	$p < 0.05$ on 3 of 5	Ours: simulated learners
Learner survey	Mean agreement about 3.0-3.3 / 5	Not conducted	Questionnaire in Appendix H

8.9 Discussion

The results confirm the three main claims of the framework within the limits of synthetic data. First, exercise-aware knowledge tracing is more accurate than DKT. Second, the integrated loop works end to end: gaps are detected, remediation and new questions are generated, the adaptive test measures progress and the knowledge state is updated until the target is reached. Third, the statistical before/after analysis can be carried out automatically. Absolute values differ from the paper because of the data and the unavailable AI models.

CHAPTER 9

Advantages, Limitations and Future Scope

9.1 Advantages

- **Personalized learning:** each learner receives plans and questions only for their own weak concepts.
- **Concept-level analysis:** mastery of each concept, not a single mark.
- **Automated question generation** from the syllabus, with duplicate removal and verification.
- **Targeted remediation** with an explanation of why each plan was given.
- **Continuous assessment:** the knowledge state is updated after every round.
- **Reduced manual effort** for teachers, who only review concepts and flagged items.
- **Aligned with OBE:** every decision uses the institution's threshold and target.
- **Robust and reproducible:** works offline, fixed seeds, 30 automated tests.

9.2 Limitations

- The evaluation uses synthetic data, not real institutional learner data.
- Only EKT and DKT are implemented; the paper also compares DKVMN, AKT and SimpleKT.
- Real LLM output could not be evaluated because the available API accounts had no credit; the offline mock produces low-quality questions.
- LLM-generated questions and plans may contain errors; teacher review of flagged items remains necessary.
- Quality depends on the uploaded course material and on the embedding model.
- EKT requires enough learner interaction data; with very few answers per concept, mastery stays near the class average.
- AI API usage has a cost per request.
- There is no user login or relational database; the prototype stores data in files.
- Only multiple-choice questions are generated; descriptive answers are not graded.
- Student data must be protected; privacy and algorithmic bias need attention in a real deployment [29].

9.3 Future Scope

- Evaluate with real OBE data and real pre- and post-tests from a live batch.
- Run the question generator with real OpenAI, Gemini and DeepSeek models and measure question quality.

- Implement the other knowledge-tracing baselines (DKVMN, AKT, SimpleKT).
- Add user accounts, a relational database and a teacher dashboard with review of flagged questions.
- Integrate with existing LMS platforms such as Moodle.
- Multimodal recommendations: videos, diagrams and interactive examples.
- Personalized learning paths across courses using prerequisite relations between concepts.
- Evaluation of descriptive answers with language models.
- Multilingual support, a mobile application and a voice-based learning assistant.

CHAPTER 10

Conclusion

This project addressed the problem that conventional assessment reports a single mark and does not identify which concepts a learner has not mastered, while manual personalization does not scale. Following the framework of Pradeesh et al. [1], we designed and implemented an AI-driven personalized learning system that processes the curriculum with Retrieval-Augmented Generation, traces each learner's knowledge with an Exercise-aware Knowledge Tracing model, identifies knowledge gaps with the OBE threshold and target, generates cross-verified remedial recommendations and new questions, and re-assesses the learner with an adaptive test in a continuous loop.

The implementation is a complete Python system with a web frontend, a REST API, a command-line interface and 30 passing automated tests. On synthetic data for a C programming course, EKT achieved a test AUC of 0.827 compared with 0.665 for DKT; a learner with five knowledge gaps reached the target on all concepts after one round of remediation; and a simulated before/after study showed higher scores on all five concepts. The project demonstrates that concept-centric evaluation combined with generative AI can provide scalable, transparent and personalized learning support aligned with Outcome-Based Education and Sustainable Development Goal 4.

References

- [1] N. Pradeesh, M. G. Thushara, K. Arun Krishna, V. Pranav and S. Krishnamoorthy, "AI-Driven Personalized Learning and Remedial Recommendation Through Knowledge Concept-Centric Evaluation," *IEEE Access*, vol. 13, pp. 207817-207837, 2025, doi: 10.1109/ACCESS.2025.3638427.
- [2] Q. Liu, Z. Huang, Y. Yin, E. Chen, H. Xiong, Y. Su and G. Hu, "EKT: Exercise-aware knowledge tracing for student performance prediction," *IEEE Transactions on Knowledge and Data Engineering*, vol. 33, no. 1, pp. 100-115, 2021.
- [3] C. Piech, J. Spencer, J. Huang, S. Ganguli, M. Sahami, L. Guibas and J. Sohl-Dickstein, "Deep knowledge tracing," in *Advances in Neural Information Processing Systems*, 2015.
- [4] J. Zhang, X. Shi, I. King and D.-Y. Yeung, "Dynamic key-value memory networks for knowledge tracing," in *Proc. 26th International Conference on World Wide Web*, 2017.
- [5] A. Ghosh, N. Heffernan and A. S. Lan, "Context-aware attentive knowledge tracing," in *Proc. 26th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2020, pp. 2330-2339.
- [6] Z. Liu, Q. Liu, J. Chen, S. Huang and W. Luo, "simpleKT: A simple but tough-to-beat baseline for knowledge tracing," in *International Conference on Learning Representations*, 2023.
- [7] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459-9474.
- [8] W. Fan et al., "A survey on RAG meeting LLMs: Towards retrieval-augmented large language models," in *Proc. 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2024, pp. 6491-6501.
- [9] C. N. Hang, C. W. Tan and P.-D. Yu, "MCQGen: A large language model-driven MCQ generator for personalized learning," *IEEE Access*, vol. 12, pp. 102261-102273, 2024.
- [10] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. EMNLP-IJCNLP*, 2019.
- [11] L. W. Anderson and D. R. Krathwohl (Eds.), *A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom's Taxonomy of Educational Objectives*. New York: Longman, 2001.
- [12] R. M. Harden, "Developments in outcome-based education," *Medical Teacher*, vol. 24, no. 2, pp. 117-120, 2002.
- [13] M. Murtaza, Y. Ahmed, J. A. Shamsi, F. Sherwani and M. Usman, "AI-based personalized e-learning systems: Issues, challenges, and solutions," *IEEE Access*, vol. 10, pp. 81323-81342, 2022.
- [14] H. Peng, S. Ma and J. M. Spector, "Personalized adaptive learning: An emerging pedagogical approach enabled by a smart learning environment," *Smart Learning Environments*, vol. 6, no. 1, 2019.
- [15] N. S. Raj and V. G. Renumol, "A systematic literature review on adaptive content recommenders in personalized learning environments from 2015 to 2020," *Journal of Computers in Education*, vol. 9, no. 1, pp. 113-148, 2022.

- [16] J. Johnson, M. Douze and H. Jegou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535-547, 2021.
- [17] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735-1780, 1997.
- [18] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *International Conference on Learning Representations*, 2015.
- [19] J. P. Kincaid, R. P. Fishburne, R. L. Rogers and B. S. Chissom, "Derivation of new readability formulas for Navy enlisted personnel," *Naval Technical Training Command, Research Branch Report 8-75*, 1975.
- [20] M. Post, "A call for clarity in reporting BLEU scores," in *Proc. Third Conference on Machine Translation*, 2018.
- [21] Y. Zhu et al., "Texygen: A benchmarking platform for text generation models," in *Proc. 41st International ACM SIGIR Conference*, 2018.
- [22] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang and M. Zhou, "MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," in *Advances in Neural Information Processing Systems*, 2020.
- [23] F. M. Lord, *Applications of Item Response Theory to Practical Testing Problems*. Hillsdale, NJ: Lawrence Erlbaum, 1980.
- [24] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, 2019.
- [25] A. T. Corbett and J. R. Anderson, "Knowledge tracing: Modeling the acquisition of procedural knowledge," *User Modeling and User-Adapted Interaction*, vol. 4, no. 4, pp. 253-278, 1994.
- [26] J. A. Hanley and B. J. McNeil, "The meaning and use of the area under a receiver operating characteristic (ROC) curve," *Radiology*, vol. 143, no. 1, pp. 29-36, 1982.
- [27] N. Tang, C. Yang, J. Fan, L. Cao, Y. Luo and A. Halevy, "VerifAI: Verified generative AI," *arXiv:2307.02796*, 2023.
- [28] M. Bond et al., "A meta systematic review of artificial intelligence in higher education: A call for increased ethics, collaboration, and rigour," *International Journal of Educational Technology in Higher Education*, vol. 21, no. 1, 2024.
- [29] O. Zawacki-Richter, V. I. Marin, M. Bond and F. Gouverneur, "Systematic review of research on artificial intelligence applications in higher education - where are the educators?," *International Journal of Educational Technology in Higher Education*, vol. 16, 2019.
- [30] FastAPI documentation, <https://fastapi.tiangolo.com>; sentence-transformers documentation, <https://www.sbert.net>; pypdf documentation, <https://pypdf.readthedocs.io>.
- [31] S. Webb, J. Holford, S. Hodge, M. Milana and R. Waller, "Lifelong learning for quality education: Exploring the neglected aspect of Sustainable Development Goal 4," *International Journal of Lifelong Education*, vol. 36, no. 5, pp. 509-511, 2017.
- [32] OpenAI API documentation, <https://platform.openai.com/docs>; Google Gemini API documentation, <https://ai.google.dev>; DeepSeek API documentation, <https://api-docs.deepseek.com>.

Appendices

Appendix A: Sample Prompts

The prompts below are the actual templates used by the system (placeholders in angle brackets).

A.1 MCQ generation prompt (qgen.py)

```
Using ONLY the course material below, write <n> distinct multiple-choice questions on the knowledge
concept
"<concept>".
Difficulty: <easy|medium|hard>. Bloom's Taxonomy level(s): <e.g. Remember / Understand>.
Learning objectives: <objectives>
Each question must have exactly 4 plausible options (distractors should reflect common
misconceptions) and one
correct answer. Vary the question structure.
Return JSON: {"questions": [{"question": str, "options": [str, str, str, str], "answer": <index
0-3>,
"bloom_level": str, "explanation": str}]}
```

COURSE MATERIAL:
<retrieved chunks>

A.2 Question verification prompt (sent to Gemini and DeepSeek)

```
You are reviewing an assessment item for the concept "<concept>" at difficulty "<level>"
(Bloom's level: <levels>).
Rate from 1 (unusable) to 5 (excellent) how well it matches the concept, difficulty and Bloom's
level, and
whether the marked answer is correct and unambiguous.
Return JSON: {"score": int, "bloom_level_ok": bool, "difficulty_ok": bool, "feedback": str}
```

A.3 Remedial recommendation prompt (remedial.py)

```
Generate a structured remedial recommendation for the concept "<concept>" for a learner who
demonstrates
<mastery>% concept mastery. The response should include five sections: Learning Objectives,
Recommended Topics
to Revise, Remedial Explanation, Two Practice Activities and Concept Gap Rationale, which will
describe the
specific weaknesses detected by the knowledge-tracing model and the reasoning behind the generated
remedial
content.
[Round 2+: The learner has NOT yet reached the target after the previous recommendation. Give a
more detailed
explanation, simpler examples and additional practice.]

Questions attempted in the most recent evaluation:
- [WRONG] (medium, Apply) Write a program to find the largest number in an array.
...
Course material: <retrieved chunks>
Return JSON: {"concept", "learning_objectives", "topics_to_revise", "remedial_explanation",
"practice_activities", "concept_gap_rationale"}
```

A.4 Concept extraction prompt (rag.py)

From the curriculum below, extract the key knowledge concepts that learners must attain. Return JSON:
 {"concepts": [{"id": "C1", "name": str, "course_outcome": "CO1", "learning_objectives": [str, ...]}]}.
 Use one concept per unit/topic.

CURRICULUM:
 <curriculum text>

Appendix B: Sample Generated MCQs

Questions taken from the generated question bank (offline generator). The correct option is marked with *.

[Algorithmic Problem Solving & Arrays | easy | Remember]
 Fill in the blank (Algorithmic Problem Solving and Array Manipulation): "To find the largest element, assume
 the first element is the largest and compare it with every other element, updating the _____
 when a
 bigger value is found."
 A. order
 B. largest
 C. search
 * D. assumption

[Algorithmic Problem Solving & Arrays | medium | Analyze]
 A learner must apply ideas from Algorithmic Problem Solving and Array Manipulation to solve a problem. Which
 statement should guide the solution?
 A. It is never necessary to consider this: dereferencing a NULL or uninitialised pointer causes undefined
 behaviour and usually a crash.
 B. A memory leak occurs when allocated memory is not no longer referenced but has not been freed.
 * C. Array elements in C are indexed from 0, so the last element of an array of size n has index n - 1.
 D. Array elements in C are not indexed from 0, so the last element of an array of size n has index n - 1.

[Algorithmic Problem Solving & Arrays | hard | Evaluate]
 Which of the following statements about Algorithmic Problem Solving and Array Manipulation is INCORRECT?
 * A. It is never necessary to consider this: a two-dimensional array stores a matrix in row-major order in C.
 B. Linear search checks each element in turn and takes time proportional to the number of elements.
 C. Bubble sort repeatedly swaps adjacent elements that are out of order until the array is sorted.
 D. Array elements in C are indexed from 0, so the last element of an array of size n has index n - 1.

[Data Structures, Memory & Files | easy | Remember]

Fill in the blank (Data Structures, Memory Management, and File Handling): "Dynamic memory is allocated on the

heap with malloc or calloc and must be released _____ with free."

- * A. explicitly
- B. structure
- C. opened
- D. accessed

Appendix C: Sample Remedial Recommendation

Recommendation generated in round 1 for learner CSE230001, concept **String Manipulation and Text Processing** (offline generator).

1. Learning objectives
 - Explain the key ideas of String Manipulation and Text Processing.
 - Apply characters correctly in small programs.
 - Identify common mistakes involving strcmp.
2. Topics to revise
 - * characters
 - The functions in ctype.h such as toupper, tolower, isalpha and isdigit classify and convert individual characters.
 - Counting vowels in a string is done by traversing the characters until the null character and testing each one.
 - * strcmp
 - The functions strlen, strcpy, strcat and strcmp are declared in the header file string.h.
 - Comparing two strings with == compares their addresses, so strcmp must be used to compare their contents.
 - * compares
 - A palindrome check compares characters from both ends of the string moving towards the middle.
 - * buffer
 - Copying a string into a buffer that is too small causes a buffer overflow, so the destination must have room for the null character.
3. Remedial explanation

The functions strlen, strcpy, strcat and strcmp are declared in the header file string.h.

Comparing two strings with == compares their addresses, so strcmp must be used to compare their contents.

Copying a string into a buffer that is too small causes a buffer overflow, so the destination must have room for the null character. The functions in ctype.h such as toupper, tolower, isalpha and isdigit classify and convert individual characters.
4. Practice activities
 - Practice 1: characters: Write and test a short program that illustrates this rule: The functions in ctype.h such as toupper, tolower, isalpha and isdigit classify and convert individual characters.
 - Practice 2: strcmp: Write and test a short program that illustrates this rule: The functions strlen, strcpy, strcat and strcmp are declared in the header file string.h.
5. Concept gap rationale

The knowledge-tracing model estimates your mastery of 'String Manipulation and Text Processing' at 35%, below the OBE target. In your latest assessment you answered 1 of 1 questions on this concept correctly.

The plan focuses on the topics that concept's questions depend on.

Cross-verification: average score 5.0 / 5 by gemini (unavailable, offline mock), deepseek (unavailable, offline mock)

Appendix D: Data Formats

D.1 OBE results CSV (first rows of current_batch.csv)

```

learner_id,exam,order,question_id,question_text,concept,course_outcome,difficulty,bloom_level,max_marks,marks
CSE230001,Internal,0,Q52,Which function opens a file for reading in C?,"Data Structures, Memory Management, and File Handling",C05,easy,Understand,2.0,0.0
CSE230001,Internal,1,Q22,Which operator is used to obtain the address of a variable?,"Operators, Pointers and Control Structures",C02,easy,Understand,2.0,1.5
CSE230001,Internal,2,Q31,How are elements of a one-dimensional array indexed in C?,"Algorithmic Problem Solving and Array Manipulation",C03,easy,Understand,2.0,2.0

```

D.2 One concept record (concepts.json)

```
{
  "id": "C1",
  "name": "Fundamentals and System Architecture",
  "course_outcome": "CO1",
  "learning_objectives": [
    "Fundamentals and System Architecture A computer system consists of the CPU, main memory, secondary storage and input/output devices connected by buses.",
    "In the von Neumann architecture both instructions and data are stored in the same main memory.",
    "The CPU repeatedly fetches an instruction from memory, decodes it and executes it, which is called the instruction cycle."
  ]
}
```

D.3 One question record (question_bank.json)

```
{
  "question": "Fill in the blank (Algorithmic Problem Solving and Array Manipulation): \n\"To find the largest element, assume the first element is the largest and compare it with every other element, updating the _____ when a bigger value is found.\",",
  "options": ["order", "largest", "search", "assumption"],
  "answer": 3,
  "concept": "Algorithmic Problem Solving and Array Manipulation",
  "difficulty": "easy",
  "bloom_level": "Remember",
  "explanation": "Based on the course material: To find the largest element, assume the first element is the largest and compare it with every other element, updating the assumption when a bigger value is found.",
  "id": "f78fb2c001"
}
```

Appendix E: API Request and Response Example

E.1 Knowledge-gap analysis (values illustrative)

```
POST /analyze
{
  "learner_id": "CSE230001",
  "records": [
    {
      "question_id": "Q33",
      "question_text": "Write a program to find the largest number in an array.",
      "concept": "Algorithmic Problem Solving and Array Manipulation",
      "difficulty": "medium",
      "max_marks": 5,
      "marks": 1.5,
      "exam": "Internal"
    }
  ]
}

Response 200
{
  "learner_id": "CSE230001",
  "concepts": [
    {
      "concept": "Algorithmic Problem Solving and Array Manipulation",
      "obe_score": 30.0,
      "mastery": 0.41,
      "level": "Beginner",
      "is_gap": true,
      "attempts": [...]
    },
    ...
  ]
}
```

Appendix F: Configuration File (config.yaml)

```
# Central configuration for the AI-Driven Personalized Learning framework.
# Values mirror the settings reported in the paper (Pradeesh et al., IEEE Access 2025).

llm:
  # Primary generator (paper Sec. V-A.1: OpenAI selected as primary engine)
  primary: openai          # openai | gemini | deepseek | mock
  # Secondary models used for cross-verification (paper Sec. III-D.3)
  verifiers: [gemini, deepseek]
  openai_model: gpt-4o-mini
  gemini_model: gemini-2.5-flash  # gemini-1.5-flash has been retired by Google
  deepseek_model: deepseek-chat
  temperature: 0.4
  # If an API key is missing, the provider silently falls back to the offline mock.
  fallback_to_mock: true

rag:
  chunk_size: 1000          # Appendix A: ~1000 chars
  chunk_overlap: 100        # Appendix A: 100-char overlap
  top_k: 4
  embedding_model: sentence-transformers/all-MiniLM-L6-v2

obe:
  threshold: 50             # Sec. III-A.1: 50-60 %
  target: 70               # ~20 % above threshold

mcq:
  generation_multiplier: 3   # Appendix A: generate ~3x the required questions
  dedup_threshold: 0.85     # Sec. III-D.1: cosine similarity cut-off
  verification_min_score: 3  # min avg verifier score (1-5) to accept a question

remedial:
  max_refinement_rounds: 2
  verification_min_score: 3

evaluation:
  # Instructor-defined rule per level: number of questions and pass percentage
  # e.g. core course "5 and 80", optional course "1 and 100" (Sec. IV-A.4)
  default_rule:
    questions_per_level: 5
    pass_percentage: 80

ekt:
  embedding_dim: 100        # Sec. V-B
  hidden_dim: 100
  dropout: 0.3
  learning_rate: 0.001
  batch_size: 32
  epochs: 12
  train_split: 0.75
  max_seq_len: 200
  use_attention: true       # EKTA variant (exercise-similarity attention)
  exercise_dropout: 0.0     # >0 hides exercise text during training (robustness to unseen questions)

paths:
  store_dir: data/store
  models_dir: data/models
```

Appendix G: Test Execution Output

```
$ python -m pytest -q
.....
30 passed
```

[100%]

Appendix H: Learner Questionnaire

Questionnaire for a future learner evaluation, based on Table 7 of the paper [1]. Each statement is rated on a five-point Likert scale (1 = strongly disagree, 5 = strongly agree). It was not administered in this project.

Table A.1: Learner questionnaire

Section	Statement
A. Usefulness and relevance	The remedial recommendations matched the areas where I needed improvement.
	The suggestions were relevant to my recent mistakes.
	The recommendations helped me focus on important concepts.
B. Clarity and quality	The recommendations were clearly written and easy to understand.
	The structure (objectives, topics, explanation, practice) made the recommendations useful.
	The practice problems were well designed and appropriate in difficulty.
C. Perceived impact	After following the recommendations, I felt more confident about the concept.
	The recommendations helped me see what I was missing.
D. Engagement and satisfaction	I found the recommendations engaging and motivating.
	I prefer these AI-generated personalized recommendations over generic feedback.
	Overall, I am satisfied with the personalized recommendation system.
E. Open-ended	What did you find most helpful? What improvements would you suggest? Other comments.

Appendix I: Installation and User Manual

```

# 1. Install (once)
python -m venv .venv
.venv\Scripts\activate          (macOS/Linux: source .venv/bin/activate)
pip install -r requirements.txt

# 2. Optional: real AI models - copy the template and add your keys
copy .env.example .env          (then edit .env; never commit it)

# 3. Run everything end to end
python -m plrs demo

# 4. Start the web application
python -m plrs serve            then open http://127.0.0.1:8000/

# 5. Use your own data
python -m plrs ingest path/to/curriculum.pdf    (review data/store/concepts.json)
python -m plrs train-kt --data historical.csv
python -m plrs analyze --data results.csv --learner <id>

# 6. Reproduce the results and run the tests
python -m plrs experiments
python -m pytest -q

```

Appendix J: Sample Command-Line Output

Table A.2: Knowledge-gap analysis output for learner CSE230001

Concept	OBE %	Mastery	Level	Gap
Algorithmic Problem Solving and Array Manipulation	58.3	0.55	Intermediate	YES
Data Structures, Memory Management, and File Handling	12.5	0.43	Beginner	YES
Fundamentals and System Architecture	28.6	0.31	Beginner	YES
Operators, Pointers and Control Structures	75.0	0.43	Beginner	YES
String Manipulation and Text Processing	75.0	0.35	Beginner	YES